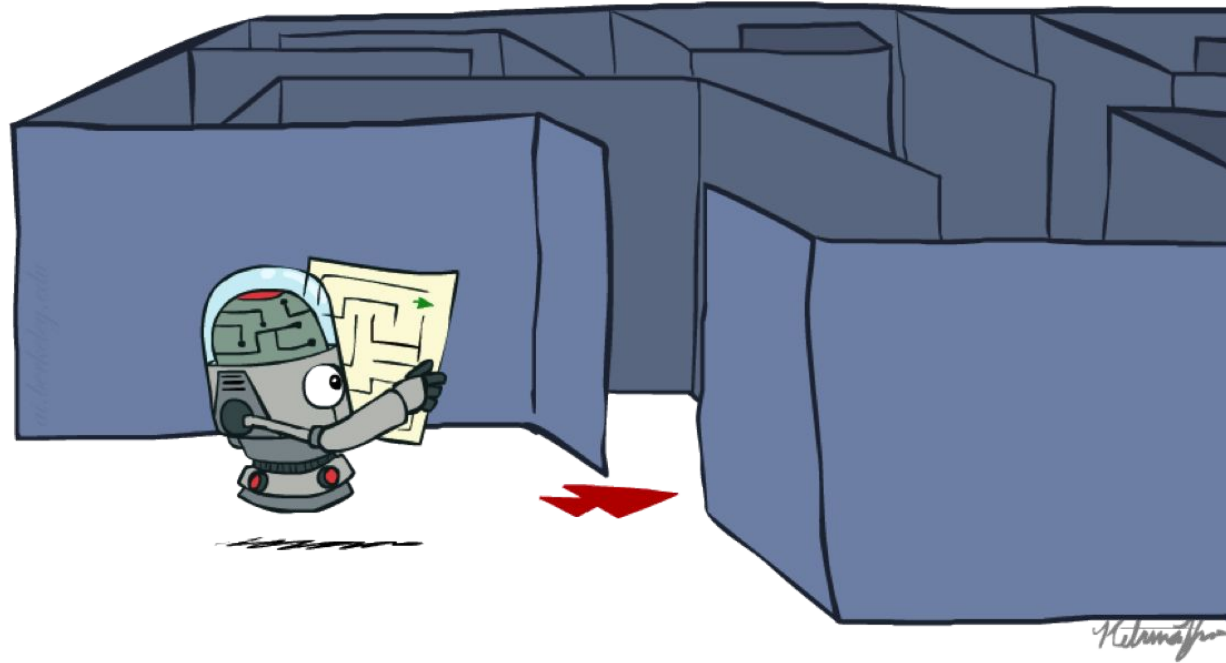


Search

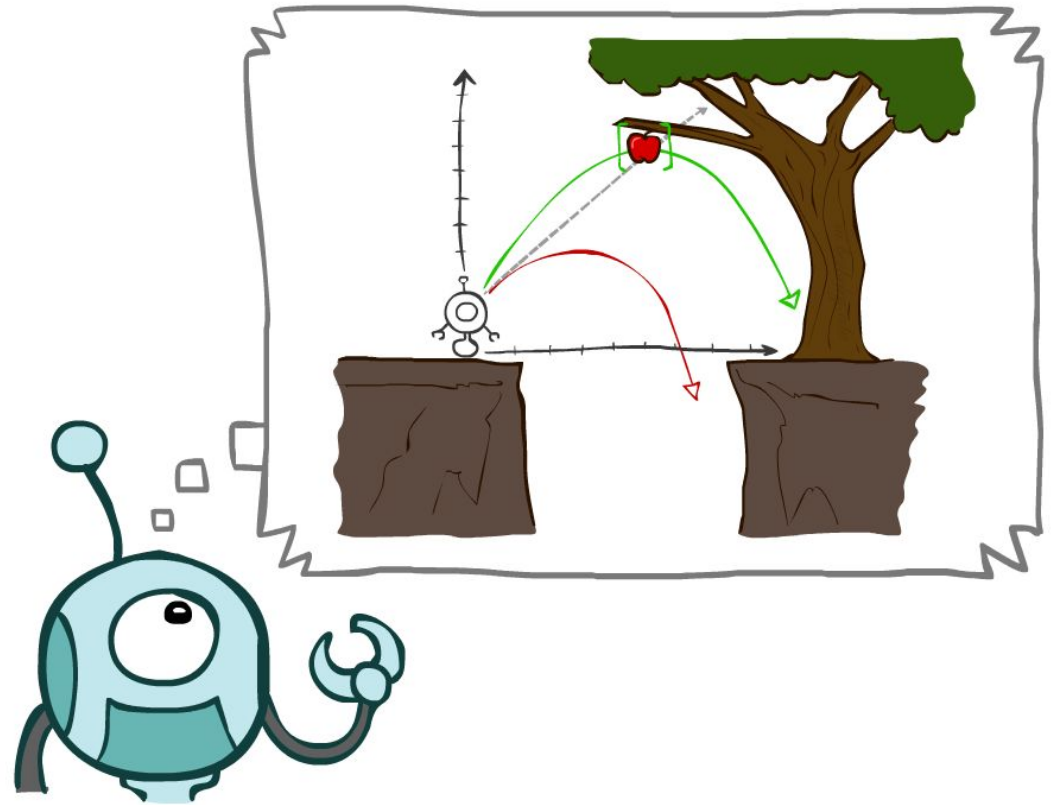


Dr. A F M Shahab Uddin

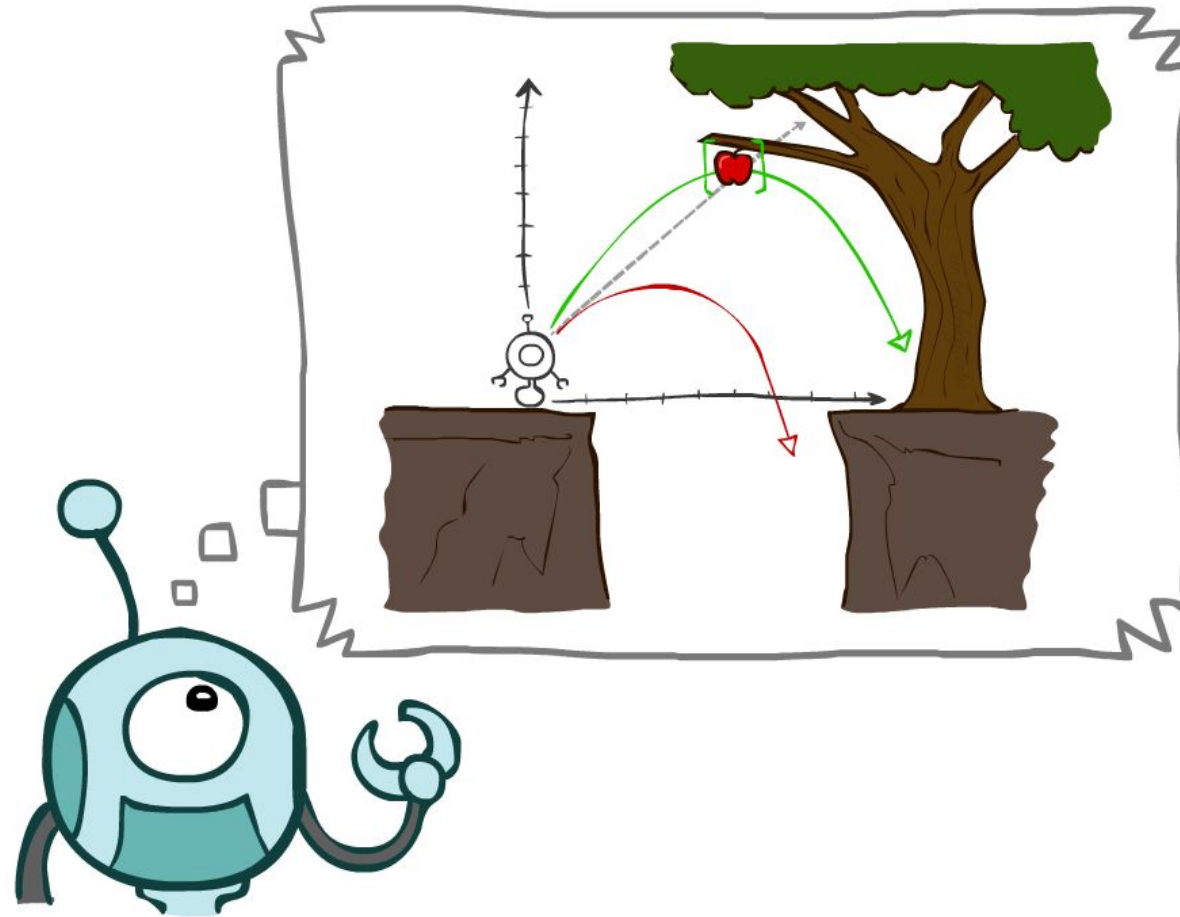
Jashore University of Science and Technology

Today

- Agents that Plan Ahead
- Search Problems
- **Uninformed Search Methods**
 - Depth-First Search
 - Breadth-First Search
 - Uniform-Cost Search



Agents that Plan



These agents think before acting. They consider the future consequences of their actions and choose the best possible sequence of steps to reach a goal.

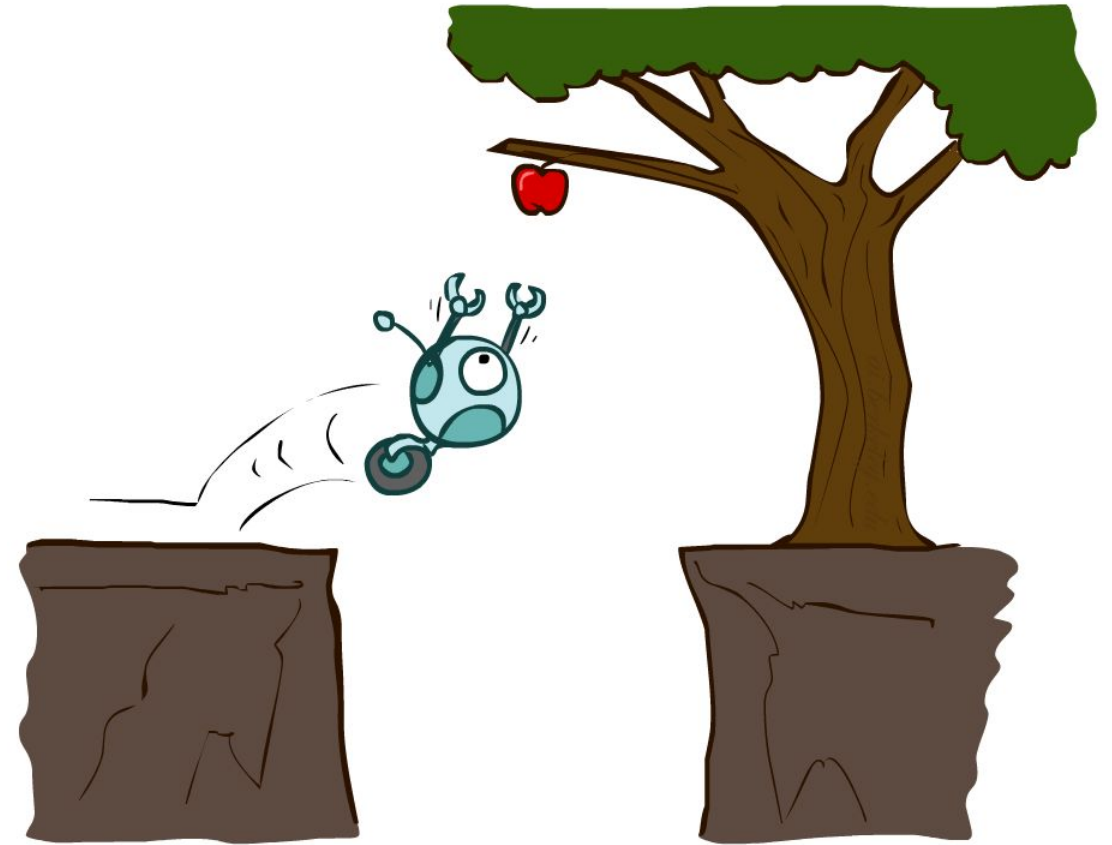
Reflex Agents

A Reflex Agent is the simplest type of intelligent agent in AI. It selects actions based only on the current percept, without considering past experiences or future consequences.

- **Reflex agents:**
 - Choose action based on current percept (and maybe memory)
 - May have memory or a model of the world's current state
 - Do not consider the future consequences of their actions
 - **Consider how the world IS**
- **Can a reflex agent be rational?**

Yes, a reflex agent can be rational if the environment is simple, fully observable, and does not require planning. In such environments, the agent can react using simple condition-action rules (like “if this happens, do that”) and still always choose the correct action.

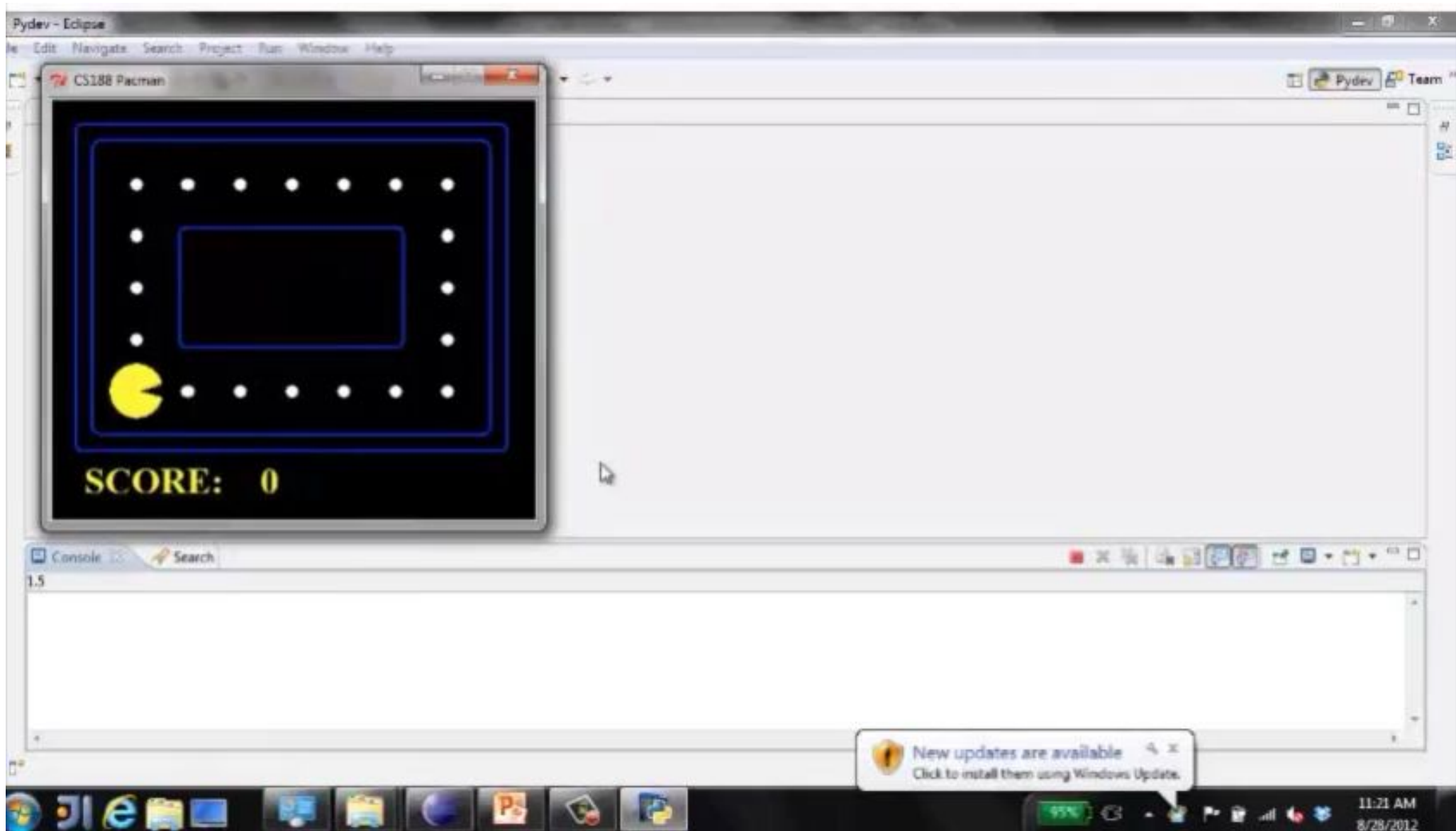
Rational: Doing the right action at the right time to achieve the best possible outcome



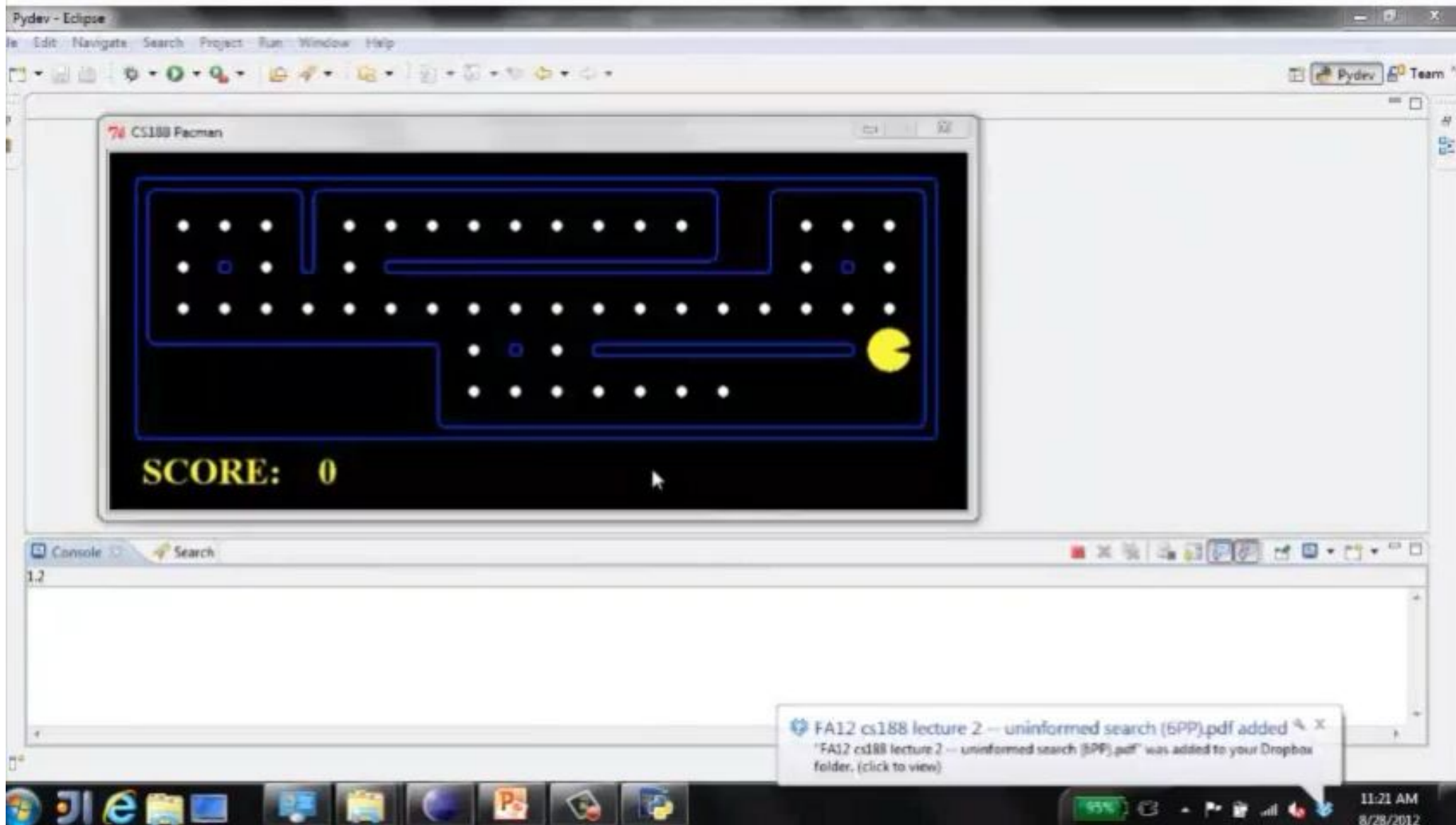
[Demo: reflex optimal (L2D1)]

[Demo: reflex optimal (L2D2)]

Video of Demo Reflex Optimal

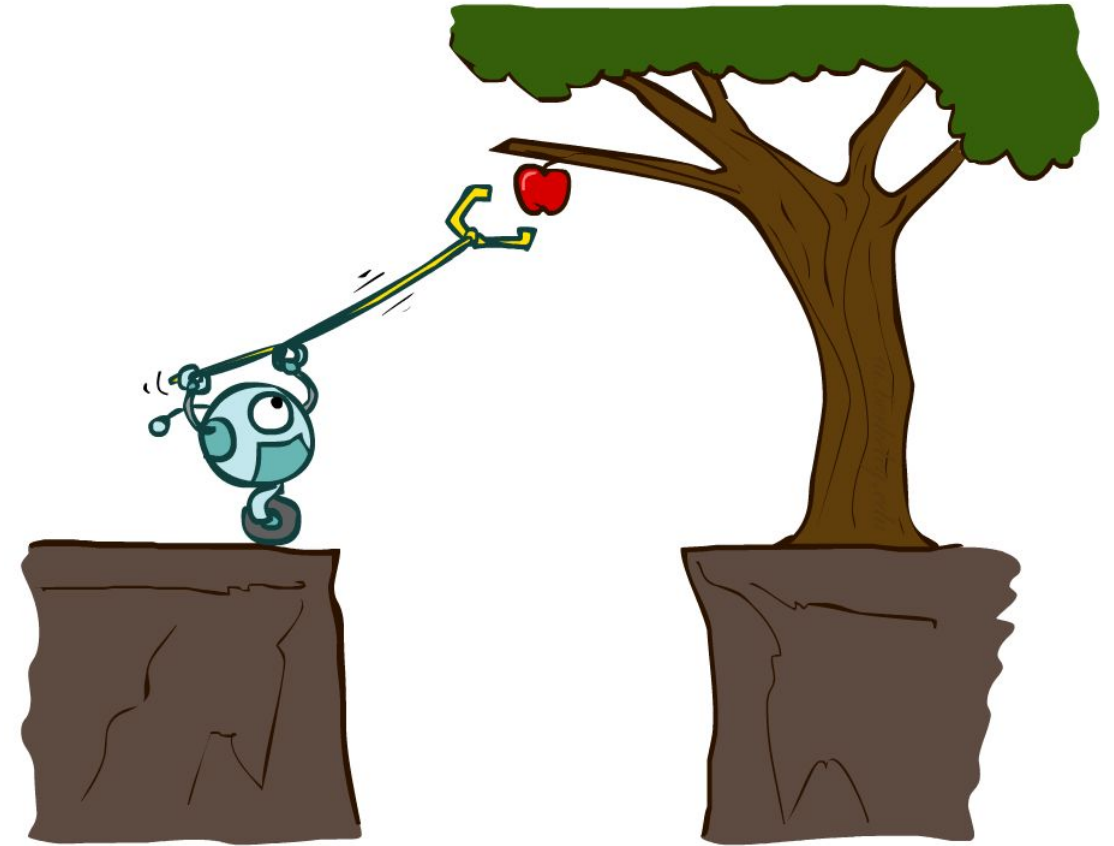


Video of Demo Reflex Odd



Planning Agents

- **Planning agents:**
 - Ask “what if”
 - Decisions based on (hypothesized) consequences of actions
 - Must have a model of how the world evolves in response to actions
 - Must formulate a goal (test)
 - **Consider how the world WOULD BE.** So A Planning Agent is an AI agent that thinks before acting. Instead of reacting immediately, it analyzes the environment, predicts future outcomes, and creates a sequence of actions (a plan) to achieve a goal in the most efficient way.



[Demo: replanning (L2D3)]

[Demo: mastermind (L2D4)]

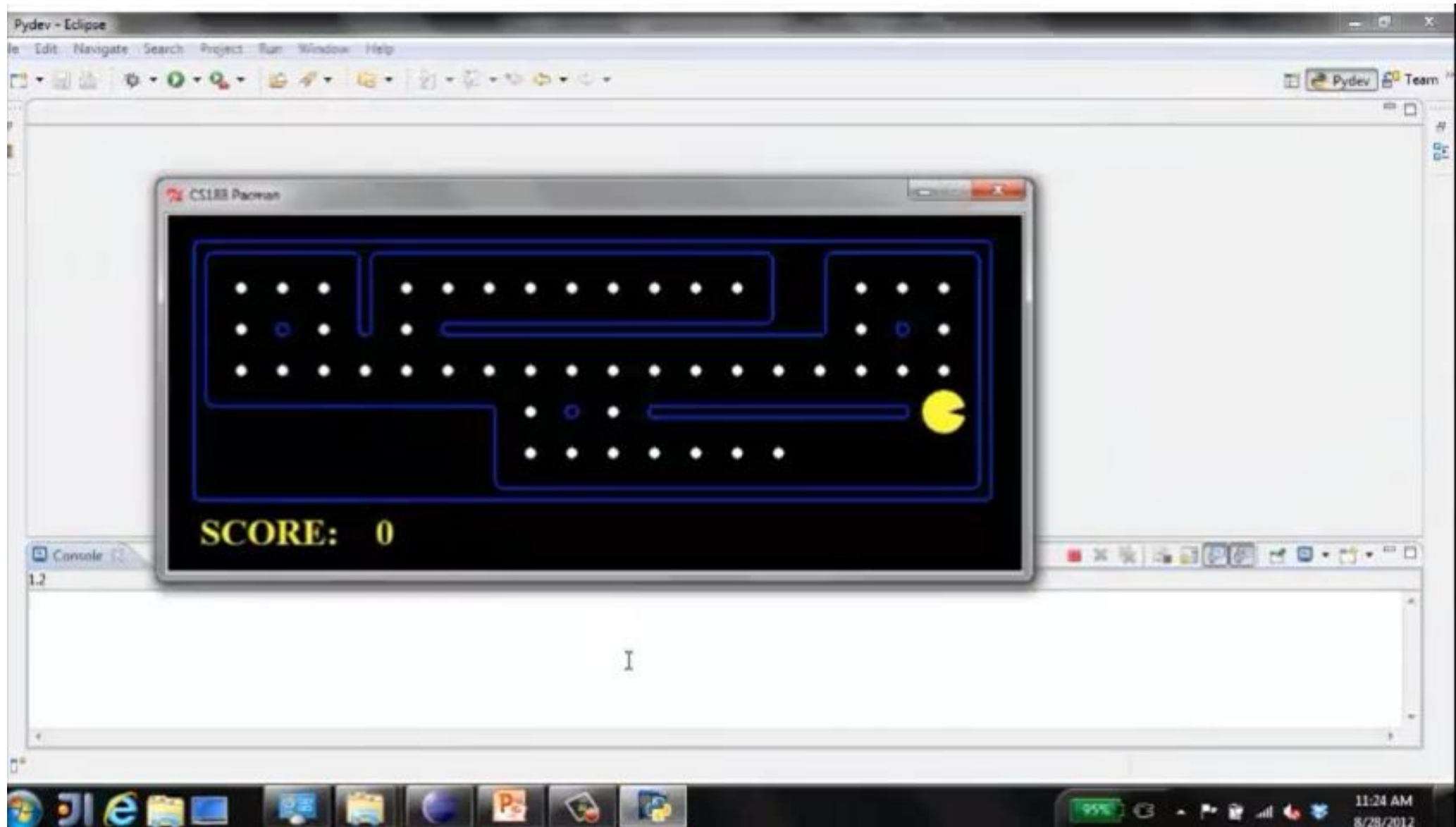
Optimal vs. Complete Planning

In AI, optimal planning finds the best possible plan according to a criterion such as the shortest path or minimum cost. It guarantees the highest-quality solution but may take more time to compute. On the other hand, complete planning guarantees that the planner will find a solution if one exists, even if the solution is not the best. Thus, optimal planning focuses on solution quality, while complete planning focuses on solution existence.

Planning vs. Replanning

Planning refers to generating a full sequence of actions before execution, assuming the environment remains stable. It is suitable for static or predictable environments. Replanning, however, occurs when the environment changes or the original plan becomes invalid. The agent updates or creates a new plan during execution. Planning is done once, while replanning is done whenever changes require a new course of action.

Video of Demo Replanning





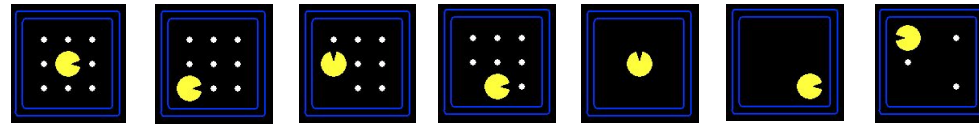
Search Problems



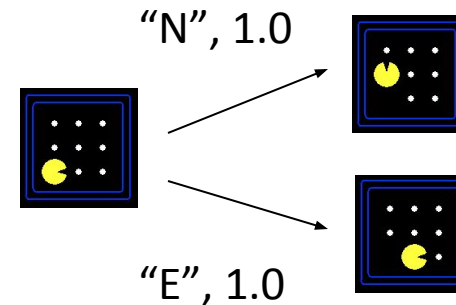
Search Problems

- A **search problem** consists of:

- A state space



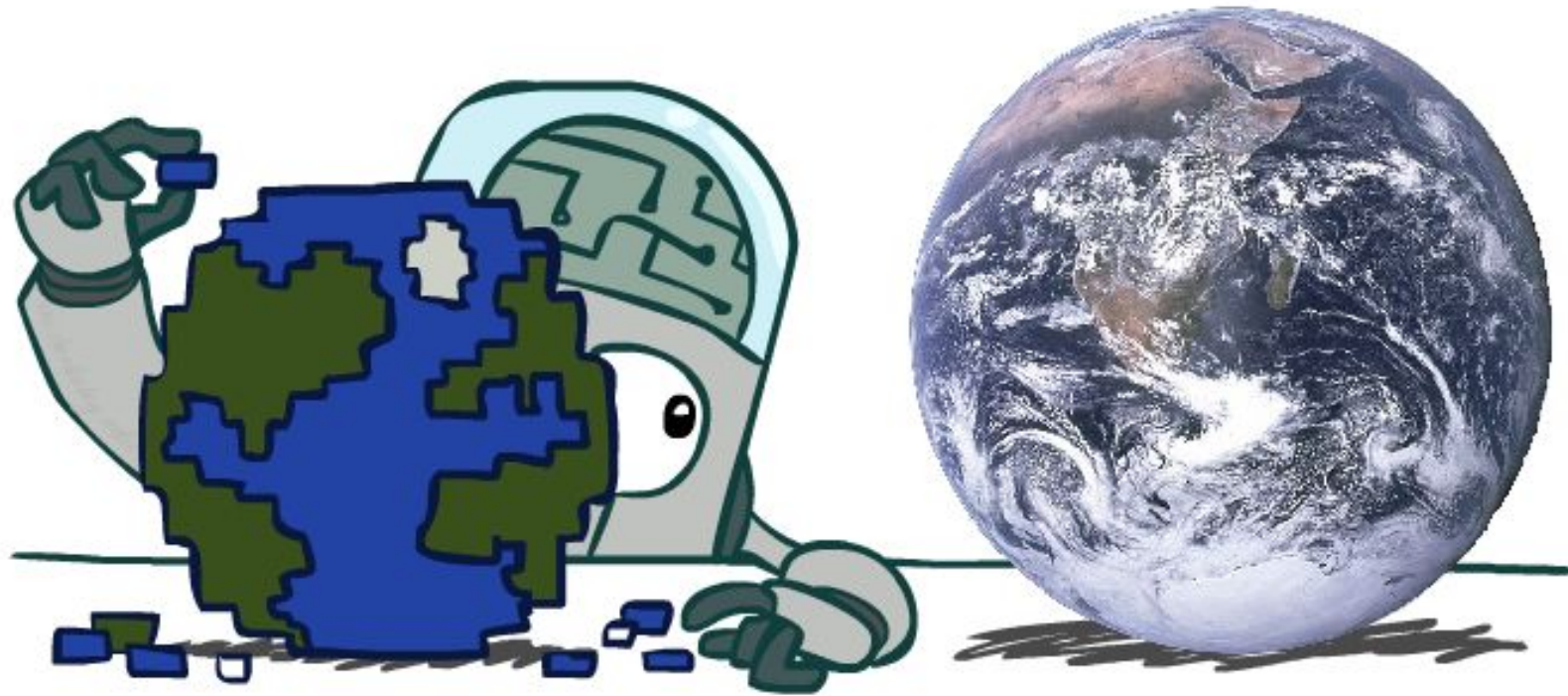
- A successor function
(with actions, costs)



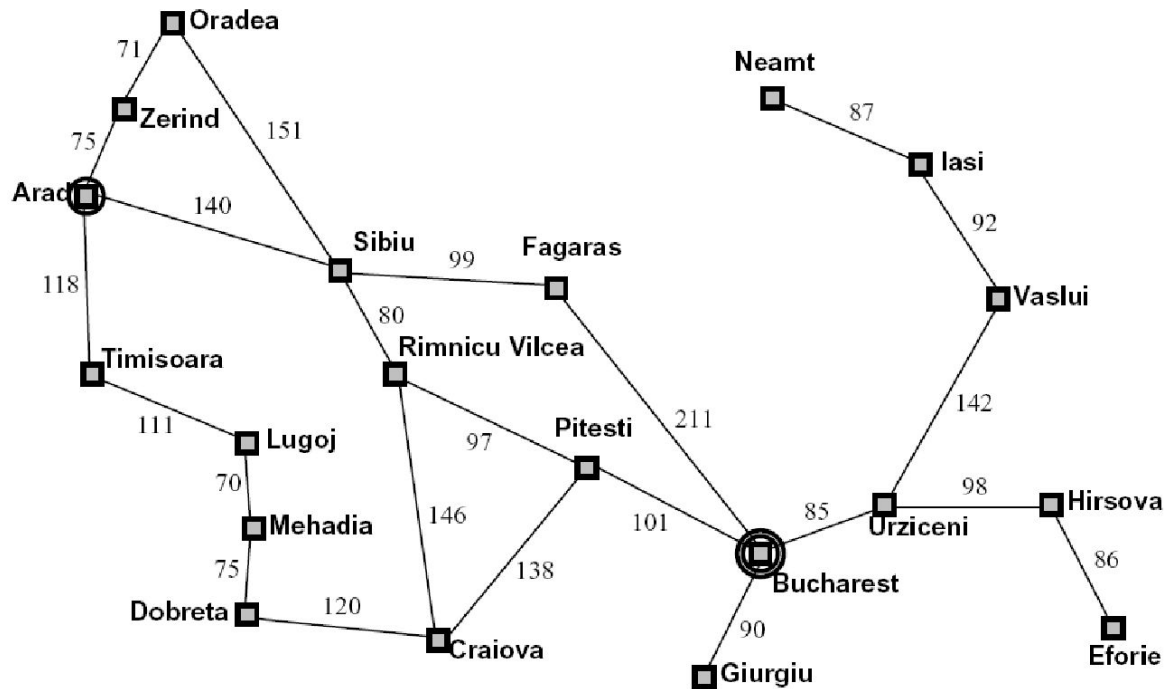
- A start state and a goal test

- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state

Search Problems Are Models



Example: Traveling in Romania

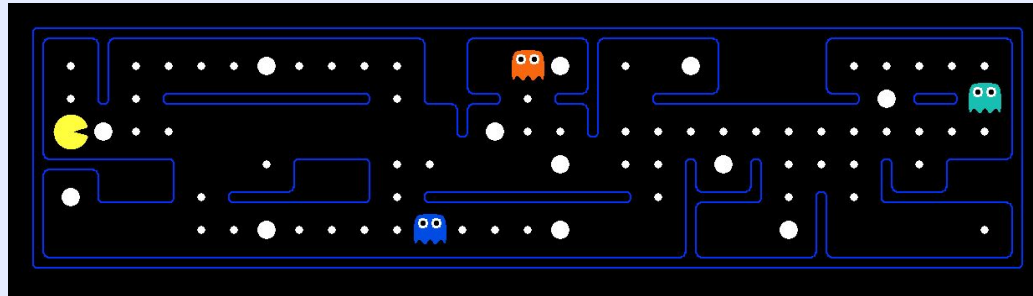


- State space:
 - Cities
- Successor function:
 - Roads: Go to adjacent city with cost = distance
- Start state:
 - Arad
- Goal test:
 - Is state == Bucharest?
- Solution?

What's in a State Space?

A state space is the complete set of all possible states (situations or configurations) that an agent can be in while solving a search problem

The **world state** includes every last detail of the environment



A search state keeps only the details needed for planning (abstraction)

■ Problem: Pathing

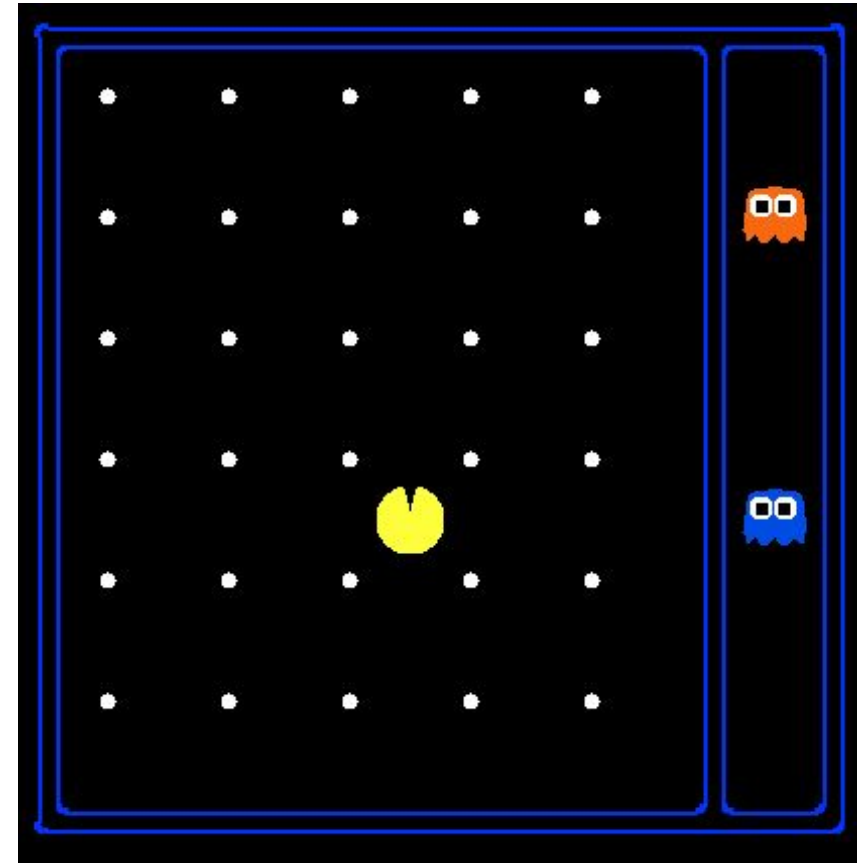
- States: (x,y) location
- Actions: NSEW
- Successor: update location only
- Goal test: is $(x,y)=\text{END}$

■ Problem: Eat-All-Dots

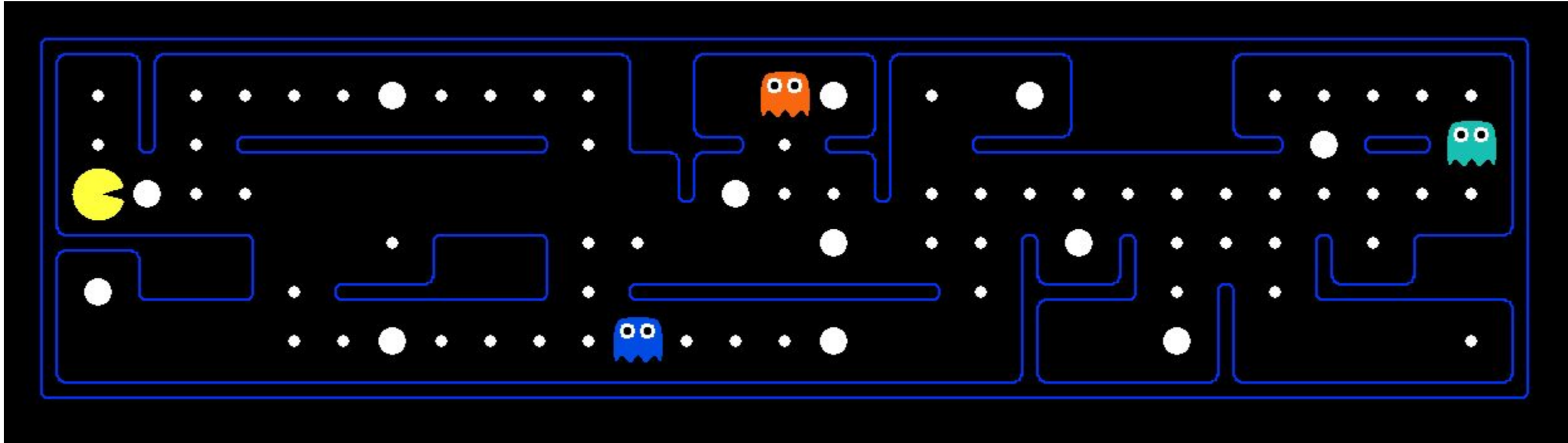
- States: $\{(x,y), \text{dot booleans}\}$
- Actions: NSEW
- Successor: update location and possibly a dot boolean
- Goal test: dots all false

State Space Sizes?

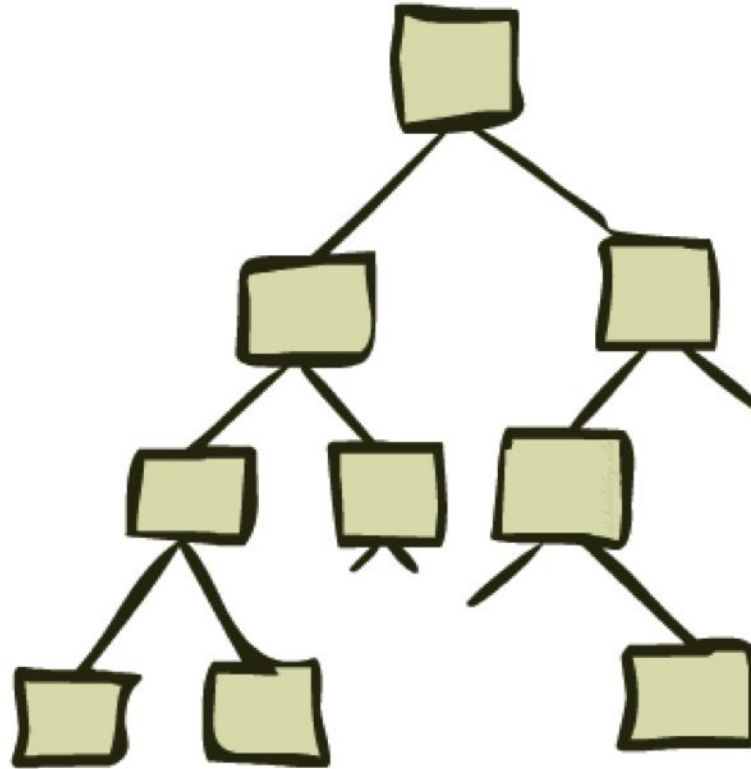
- World state:
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- How many
 - World states?
 $120 \times (2^{30}) \times (12^2) \times 4$
 - States for pathing?
120
 - States for eat-all-dots?
 $120 \times (2^{30})$



Quiz: Safe Passage

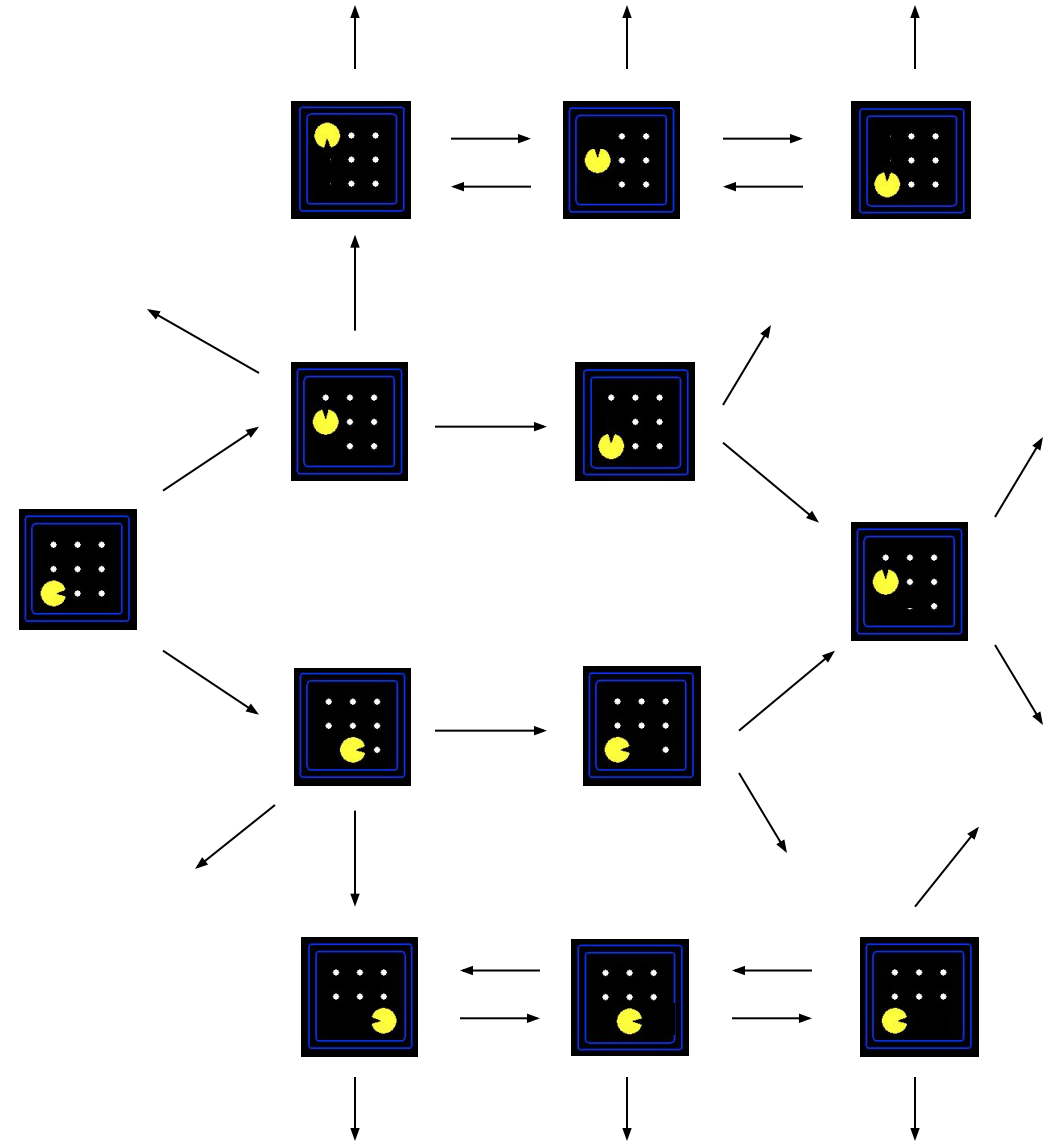


- Problem: eat all dots while keeping the ghosts perma-scared
- What does the state space have to specify?
 - (agent position, dot booleans, power pellet booleans, remaining scared time)



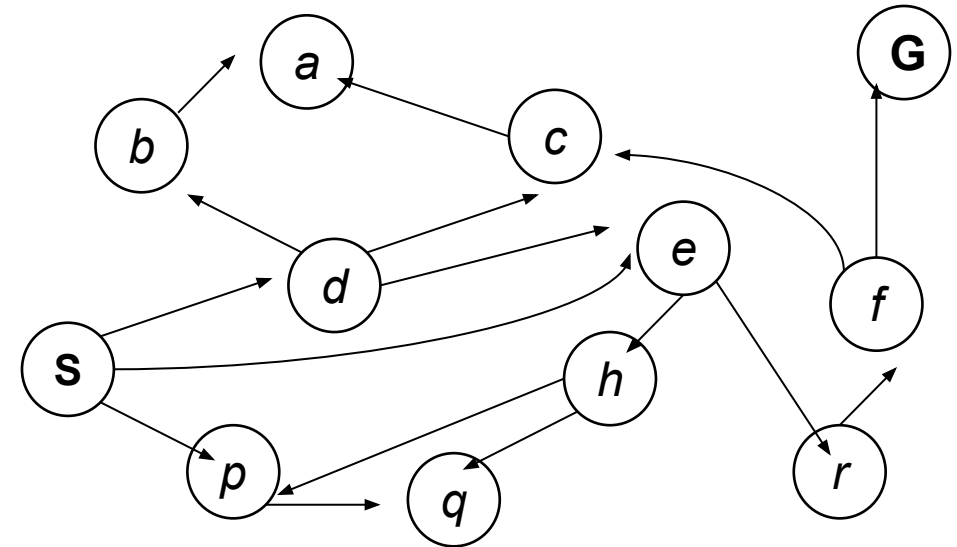
State Space Graphs

- **State space graph:** A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



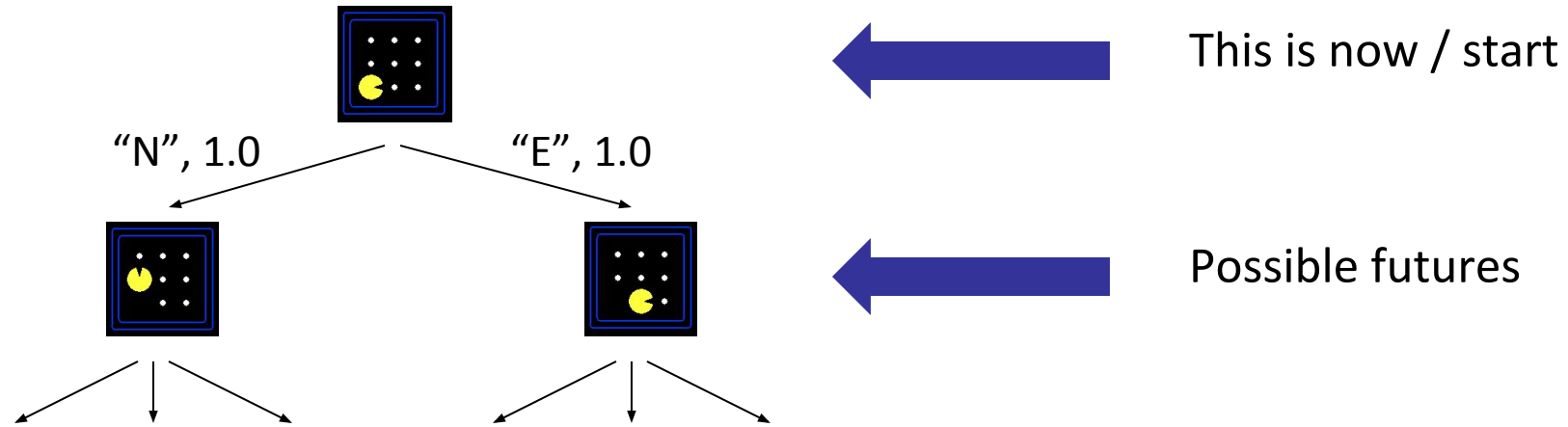
State Space Graphs

- **State space graph:** A state space graph is a mathematical model used to represent a search problem in AI. In this graph, nodes represent different states or configurations of the world, and arcs (edges) represent the actions that connect one state to another (successor states). The initial state is the starting node, and the goal test identifies one or more goal nodes that satisfy the desired condition. The search algorithm explores this graph to find a path from the initial state to a goal state.



Tiny search graph for a tiny search problem

Search Trees

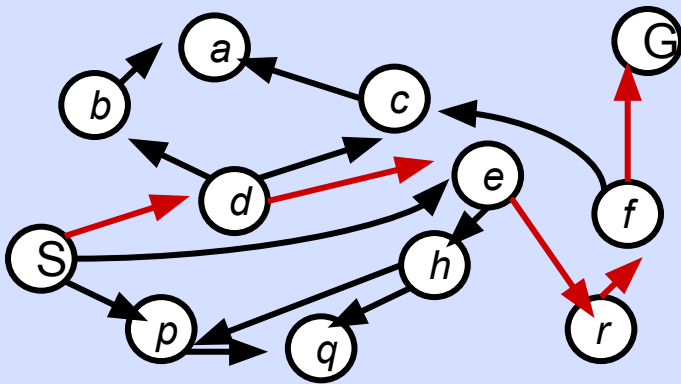


- **A search tree:**

A search tree is a tree structure used in AI to represent all possible sequences of actions an agent can take from the initial state. The root of the tree is the initial state, and each child node represents a new state reached by applying an action. As the tree expands, it shows all possible paths the agent can follow to reach a goal state. Search algorithms like BFS, DFS, and Uniform- Cost Search operate by expanding nodes in the search tree.

State Space Graphs vs. Search Trees

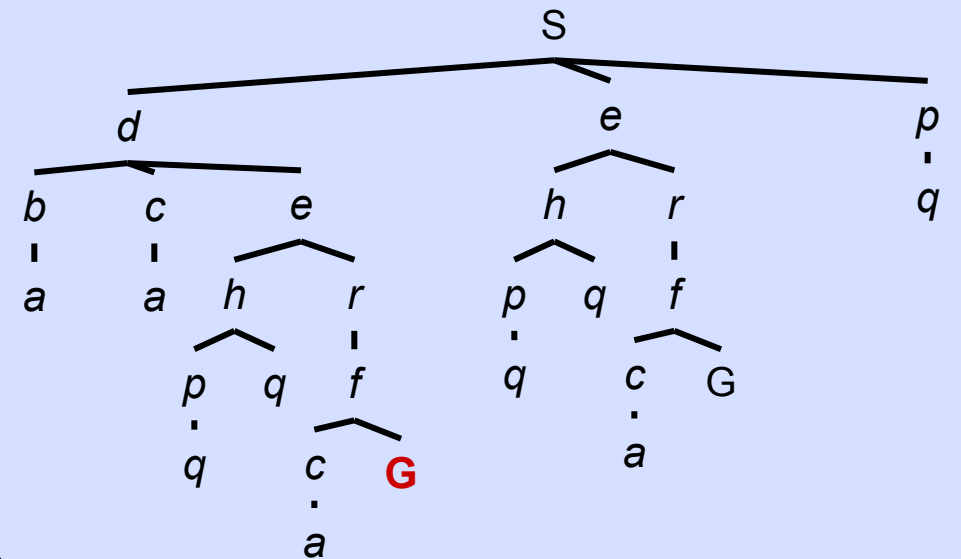
State Space Graph



Each NODE in the search tree is an entire PATH in the state space graph.

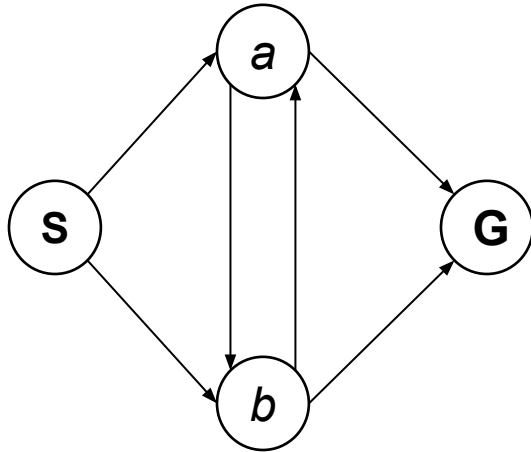
We construct both on demand – and we construct as little as possible.

Search Tree



Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:



How big is its search tree (from S)?



Important: Lots of repeated structure in the search tree!

State Space Graphs vs. Search Trees

State Space Graph

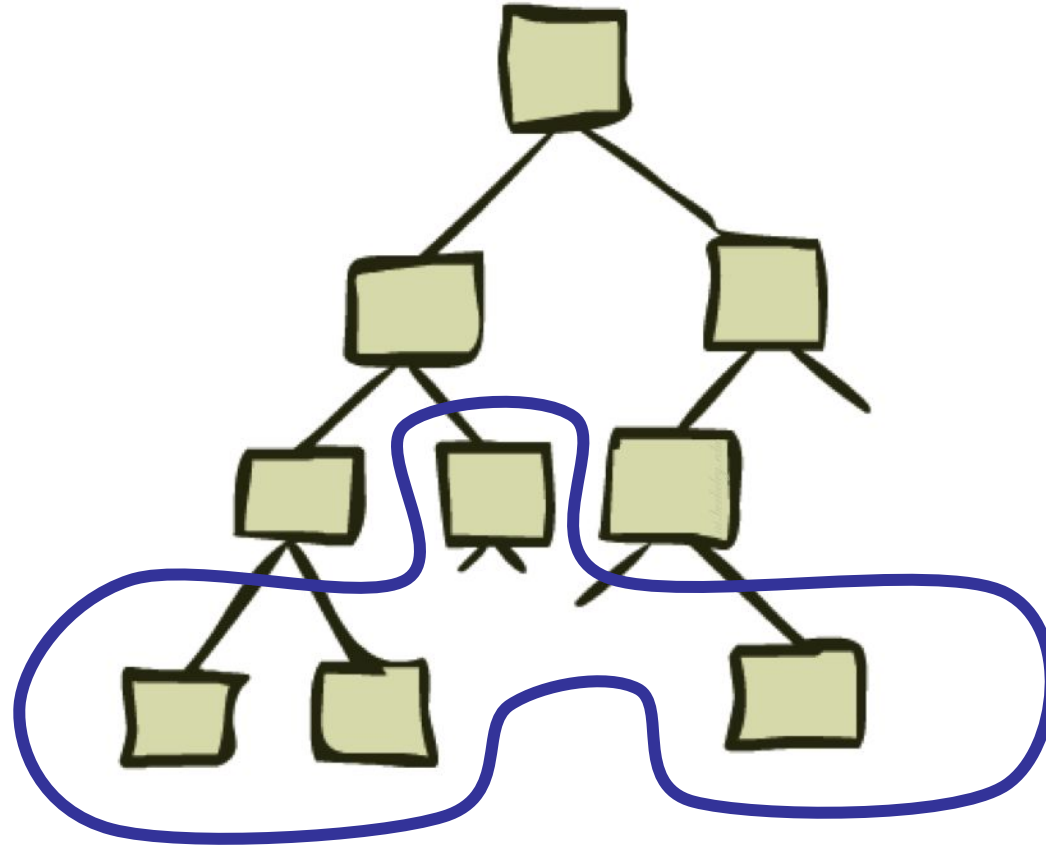
A state space graph is a mathematical representation of a search problem where each node represents a unique state and the edges show the actions that move the agent between states. In this graph, a state appears only once even if several paths can reach it, and the graph may contain loops. It describes the entire structure of the problem before the search begins.

Search Tree

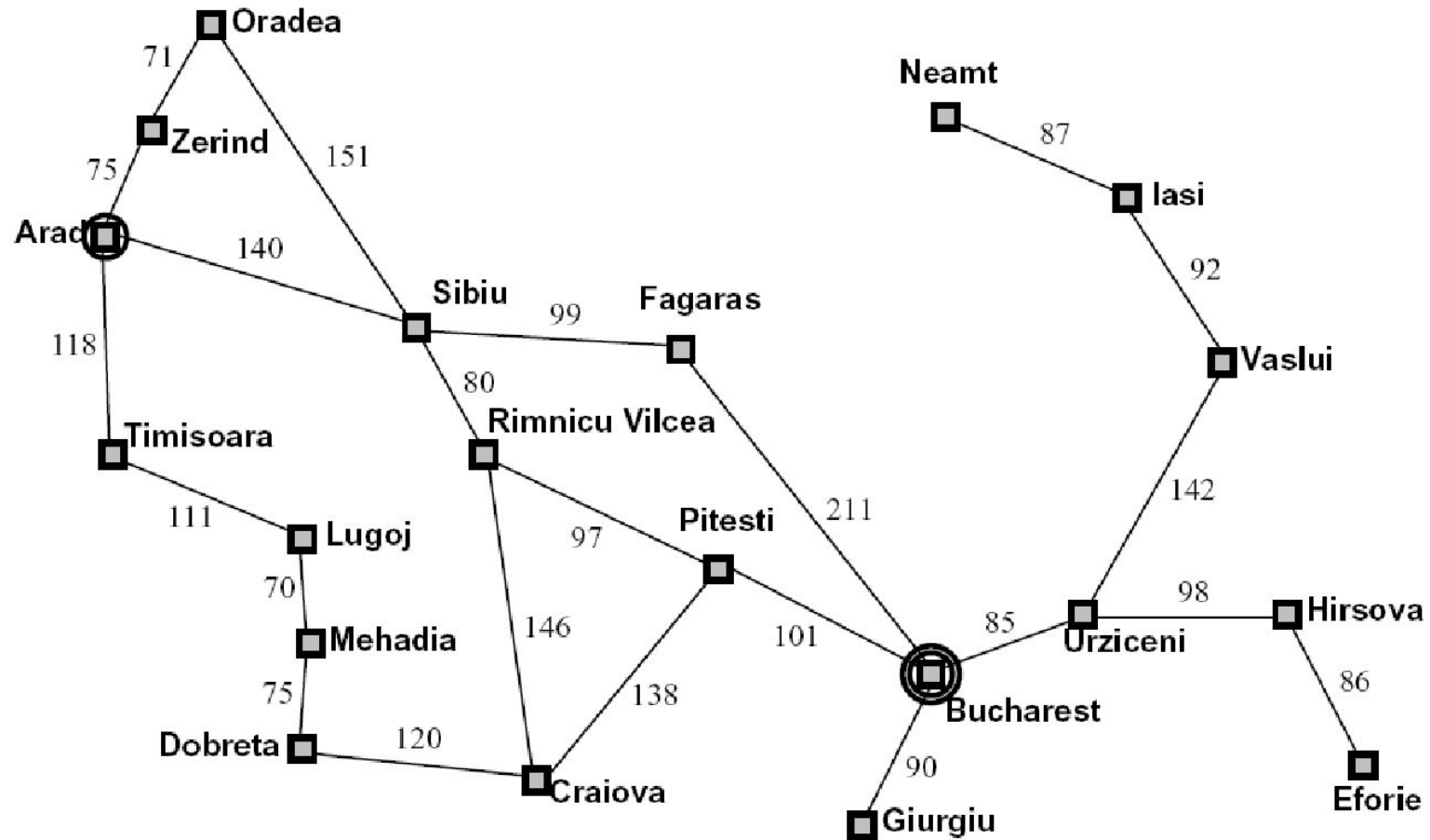
A search tree is the structure created by a search algorithm while exploring the state space. It starts with the initial state as the root, and each child node represents a successor state generated by an action. The same state can appear multiple times in the search tree if reached through different paths, and it never contains cycles. It shows how the algorithm actually explores the states

A successor state is the next place or situation you get to after doing an action.

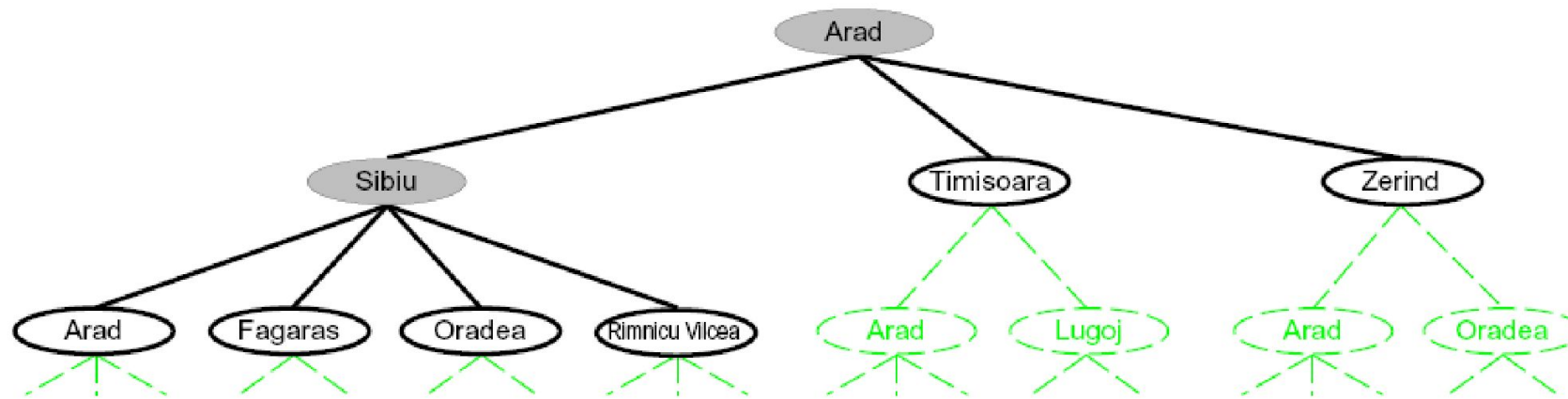
Tree Search



Search Example: Romania



Searching with a Search Tree



■ Search:

- Expand out potential plans (tree nodes)
- Maintain a **fringe** of partial plans under consideration
- Try to expand as few tree nodes as possible

General Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Important ideas:

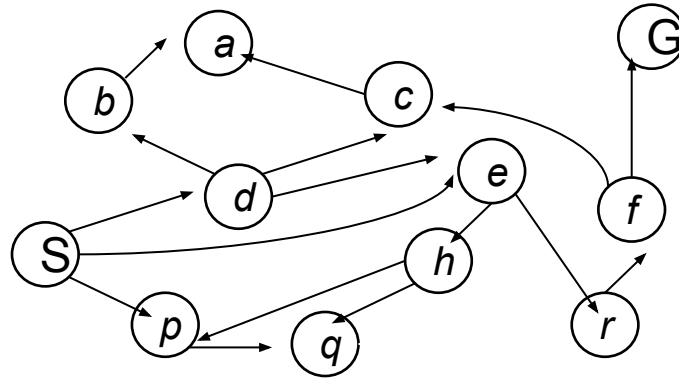
Fringe: List of unexpanded nodes.

Expansion: Generating successors of a node.

Exploration strategy: Rule for choosing which fringe node to explore next.

Main question: Which fringe node should be expanded now? — this choice defines the search algorithm.

Example: Tree Search



Depth-First Search

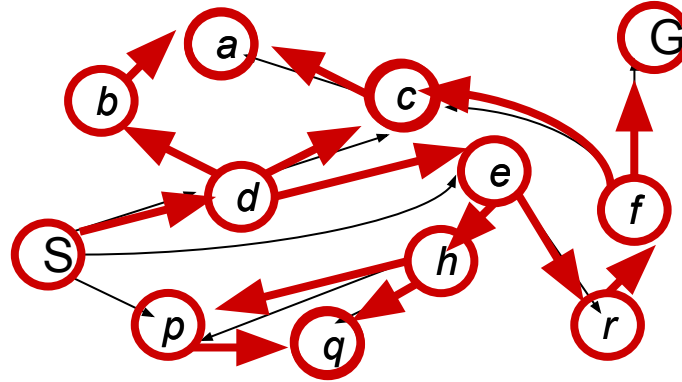


Depth-First Search

Strategy: expand a deepest node first

Implementation:

Fringe is a LIFO stack



DFS expands the deepest node first.

How it works:

Goes down one path completely

If dead end → backtracks

Uses a stack (LIFO)

Advantages:

Low memory usage

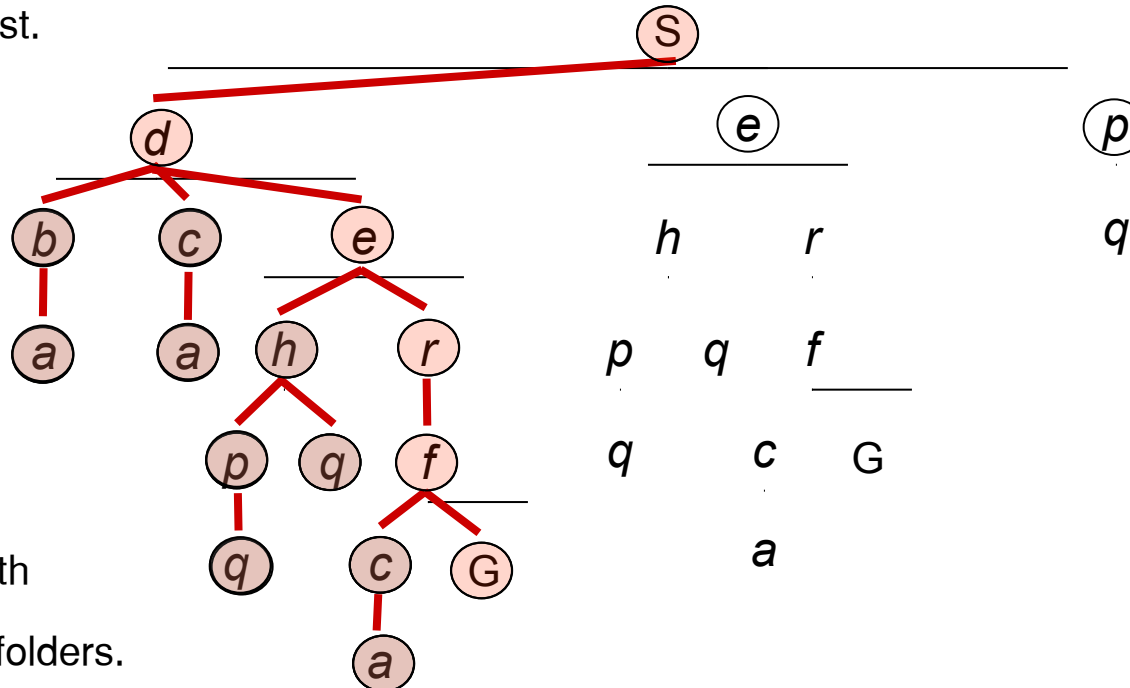
Good for deep solutions

Disadvantages:

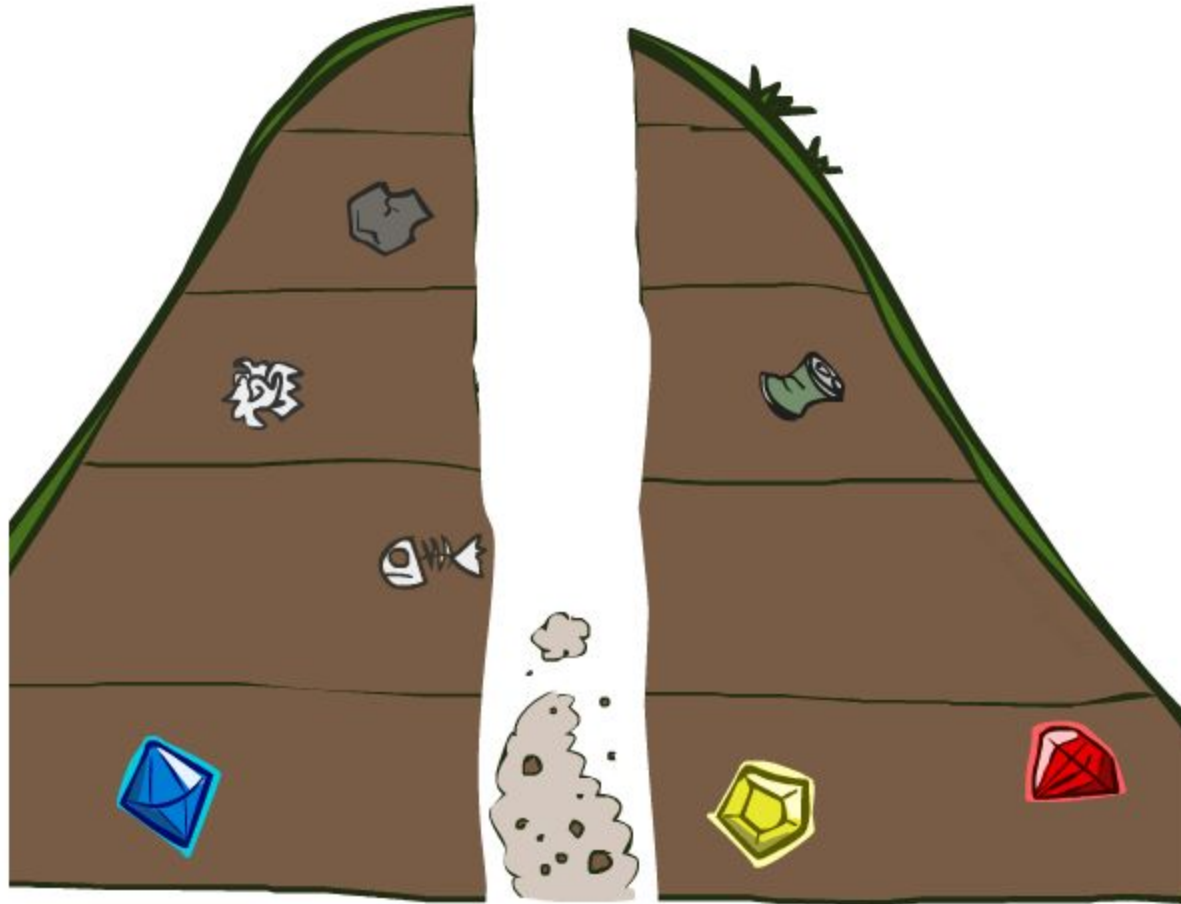
Can get stuck in infinite depth

Not guaranteed to find shortest path

Example: Exploring folders inside folders.



Search Algorithm Properties



Search Algorithm Properties

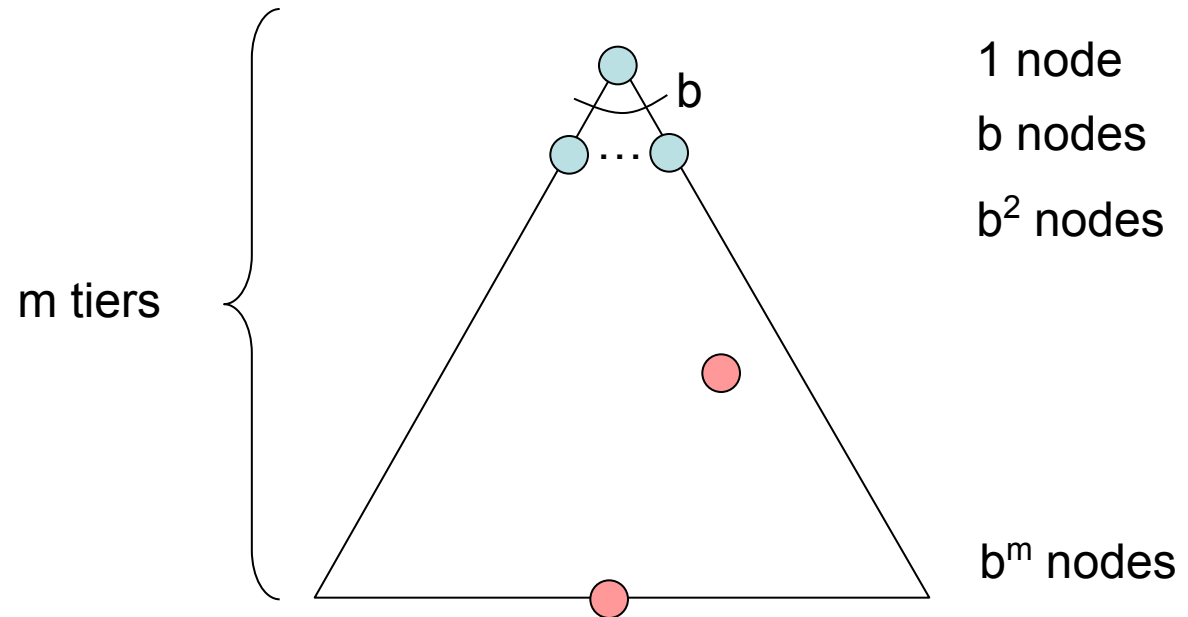
- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?

- Cartoon of search tree:

- b is the branching factor
- m is the maximum depth
- solutions at various depths

- Number of nodes in entire tree?

- $1 + b + b^2 + \dots + b^m = O(b^m)$



Depth-First Search (DFS) Properties

- **What nodes DFS expand?** DFS (Depth-First Search) always starts from the left side and keeps going deeper and deeper before looking at other paths. It follows one path as far as it can before coming back. If the solution is not on the left side, DFS may end up exploring almost the entire tree before finding the answer.

- If m is finite, takes time $O(b^m)$

- How much space does the fringe take?

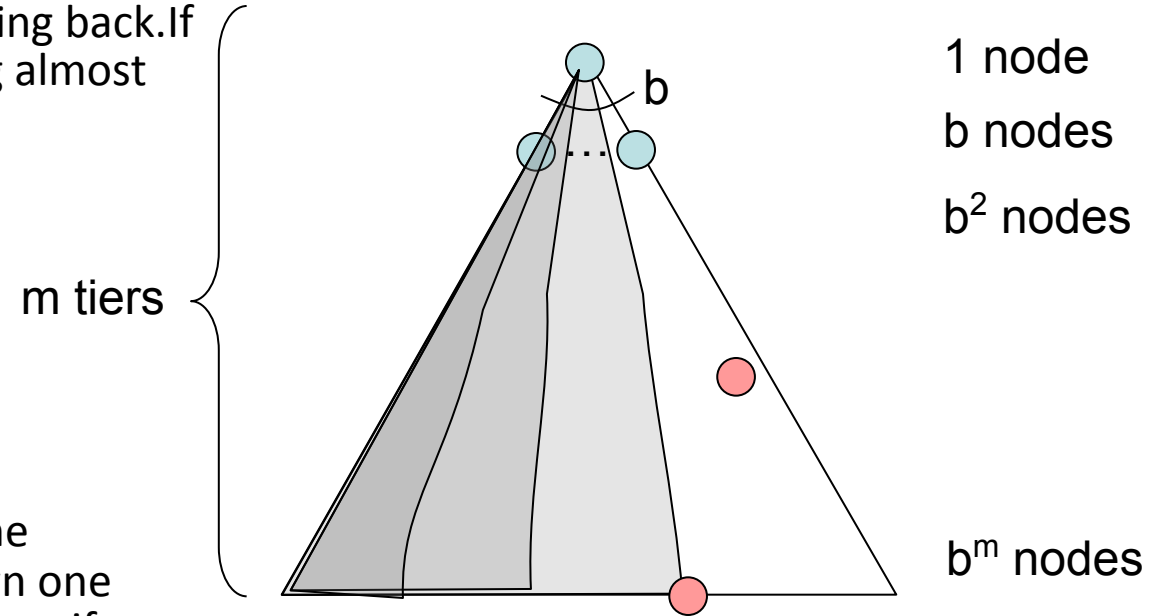
- Only has siblings on path to root, so $O(bm)$

- Is it complete?

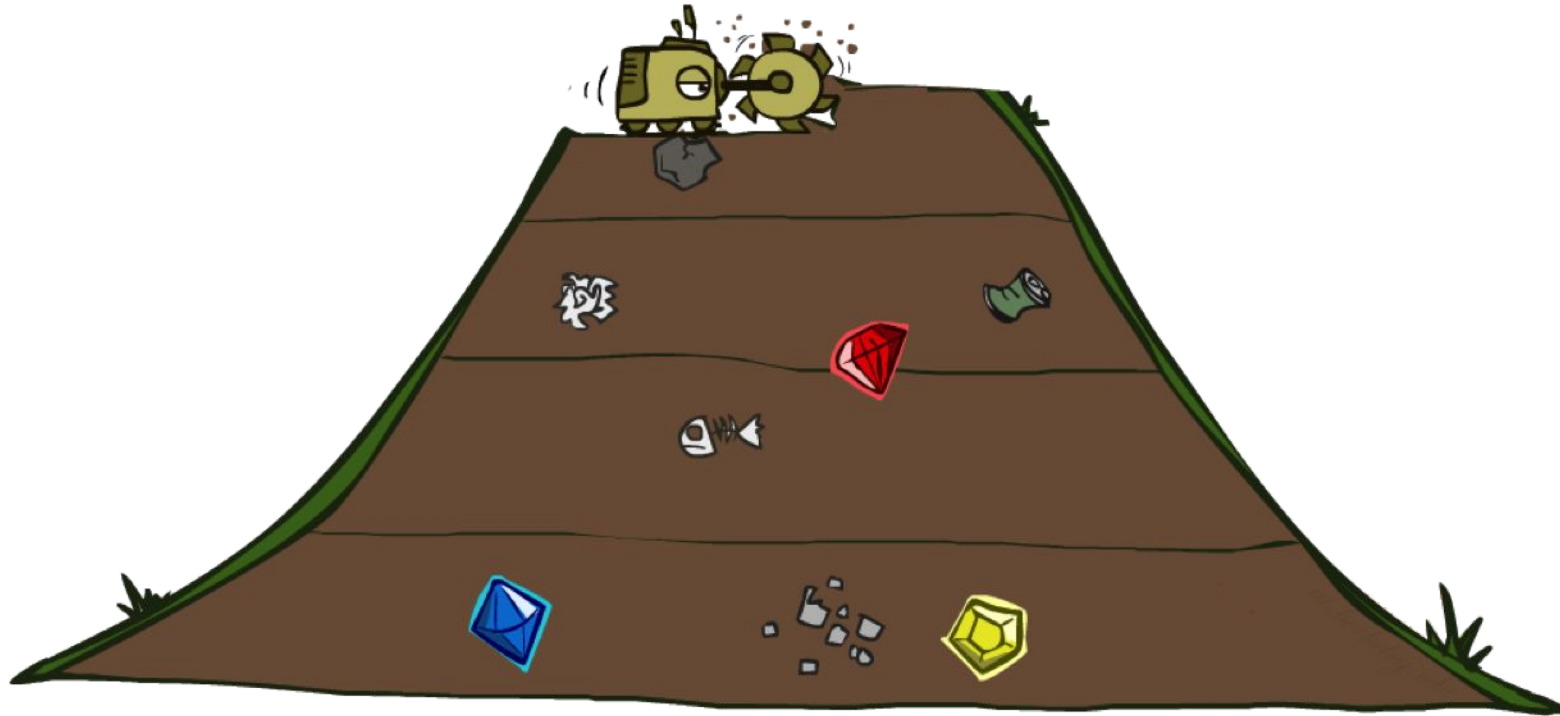
- No, m can be infinite, so DFS is not complete when the search tree can be infinite. DFS might keep going down one path forever and never come back to find the goal, even if the goal exists somewhere else in the tree.

- Is it optimal?

- No, it finds the “leftmost” solution, regardless of depth or cost



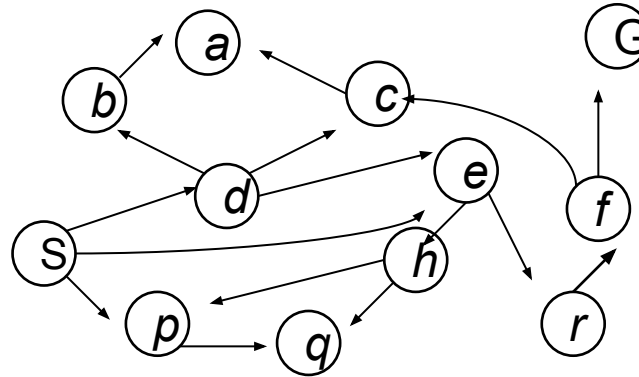
Breadth-First Search



Breadth-First Search

Strategy: expand a shallowest node first

Implementation: Fringe is a FIFO queue

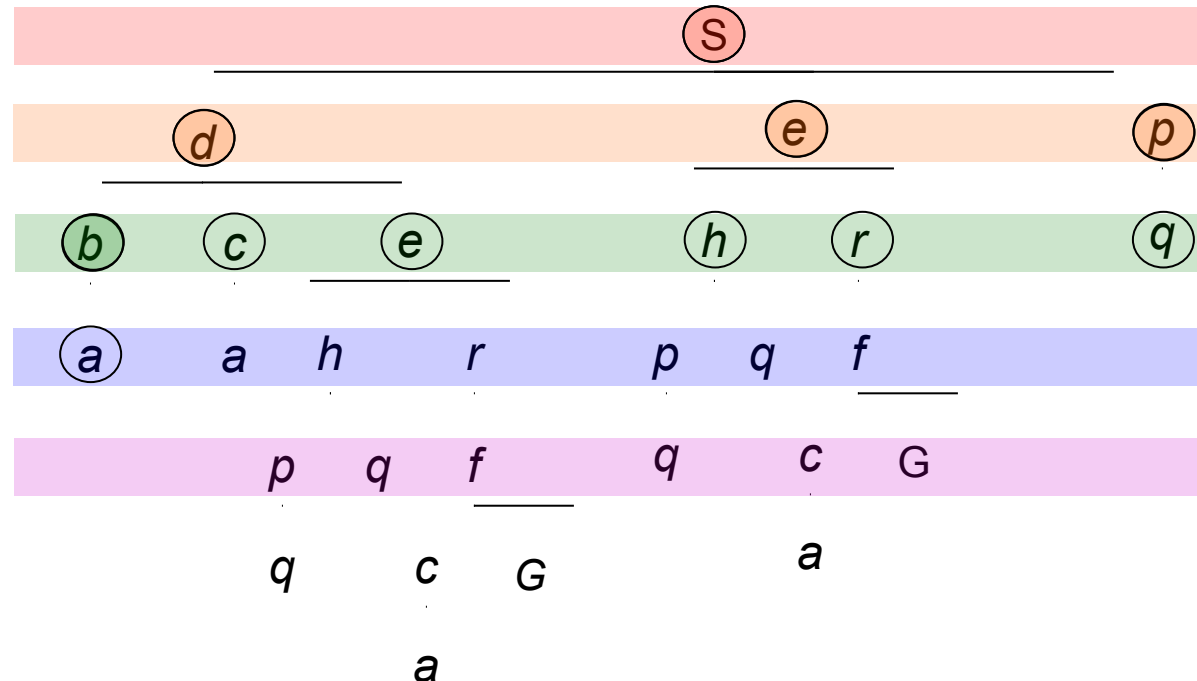


BFS expands the shallowest node first.

How it works:

Explores all neighbors first
Then moves to the next level
Uses a queue (FIFO)

Search
Tiers



Advantages:

Always finds the shortest path
Complete (will find a solution if exists)

Disadvantages:

High memory usage

Example: Finding shortest path in an unweighted graph.

Breadth-First Search (BFS) Properties

- What nodes does BFS expand?

- BFS expands nodes level by level. It will visit every node at depth 0, 1, 2, ... (up to $s-1$) before it reaches the shallowest solution at depth s .

- Search takes time $O(b^s)$

- How much space does the fringe take?

- Has roughly the last tier, so $O(b^s)$

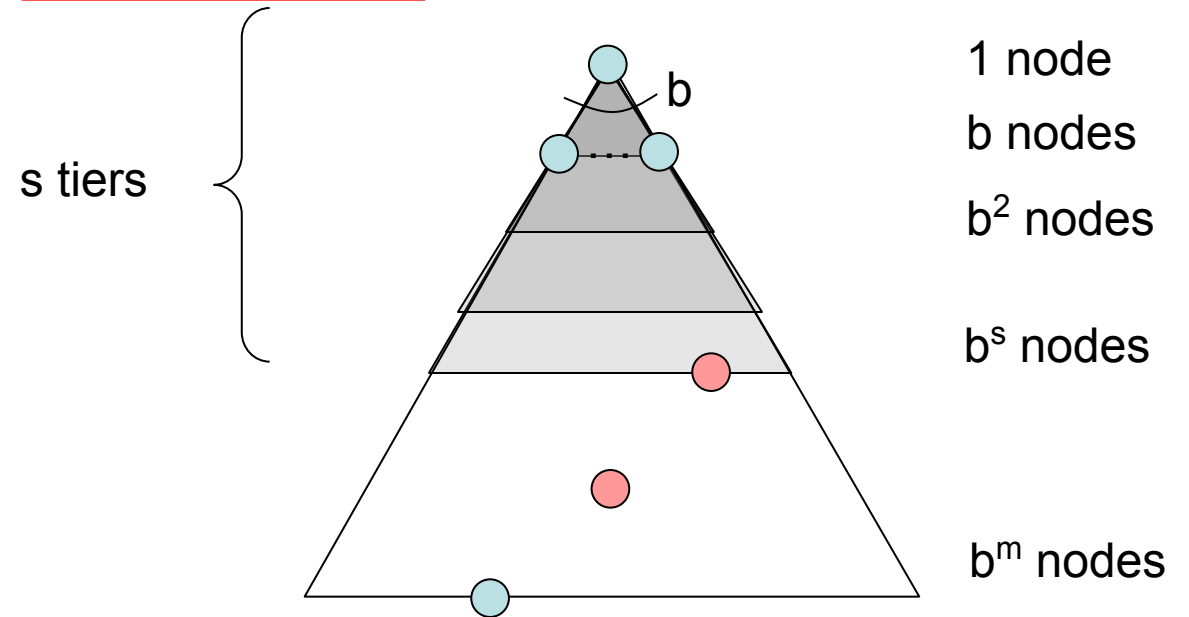
- Is it complete?

- If a solution exists, and the branching factor is finite, s will be finite, so BFS will always find the solution. Therefore, BFS is complete.

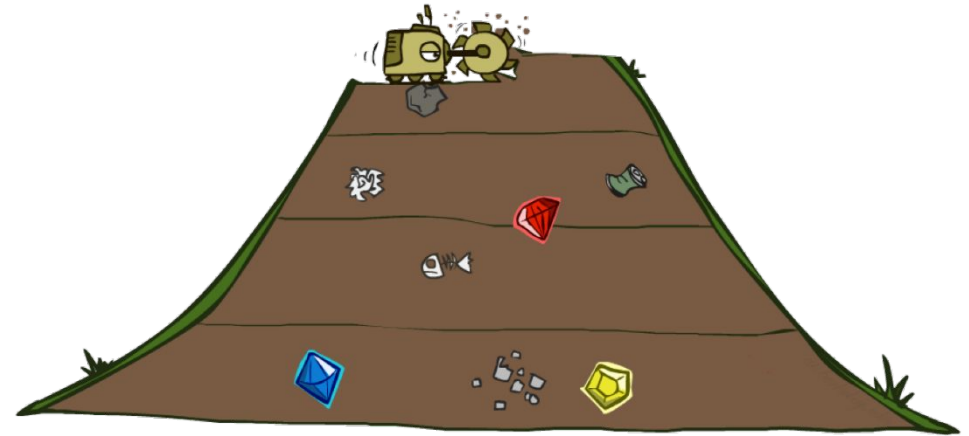
- Is it optimal?

- BFS guarantees the shortest path only when all actions have cost 1. If actions have different costs, BFS may find a path that is not optimal.

The shallowest goal node is the goal node with the minimum depth from the root.



Quiz: DFS vs BFS



Quiz: DFS vs BFS

- When will BFS outperform DFS?

BFS is better when the solution is shallow and we need the shortest path, because it explores level by level.

- When will DFS outperform BFS?

DFS is better when the solution is deep or when memory is limited, because it uses much less space.

Video of Demo Maze Water DFS/BFS (part 1)

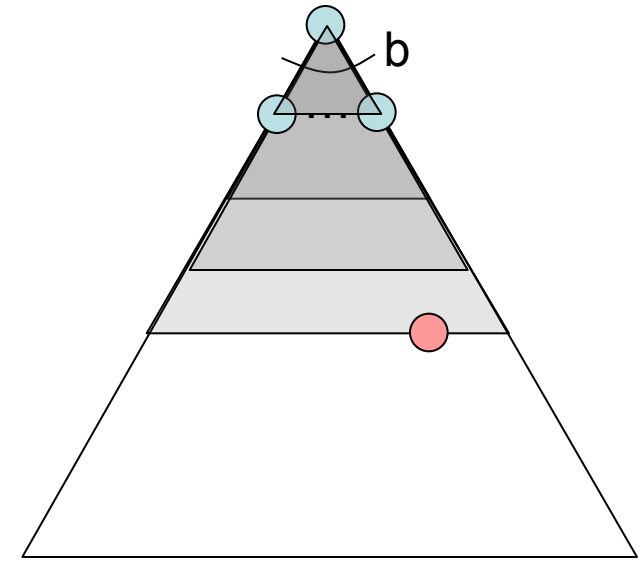


Video of Demo Maze Water DFS/BFS (part 2)

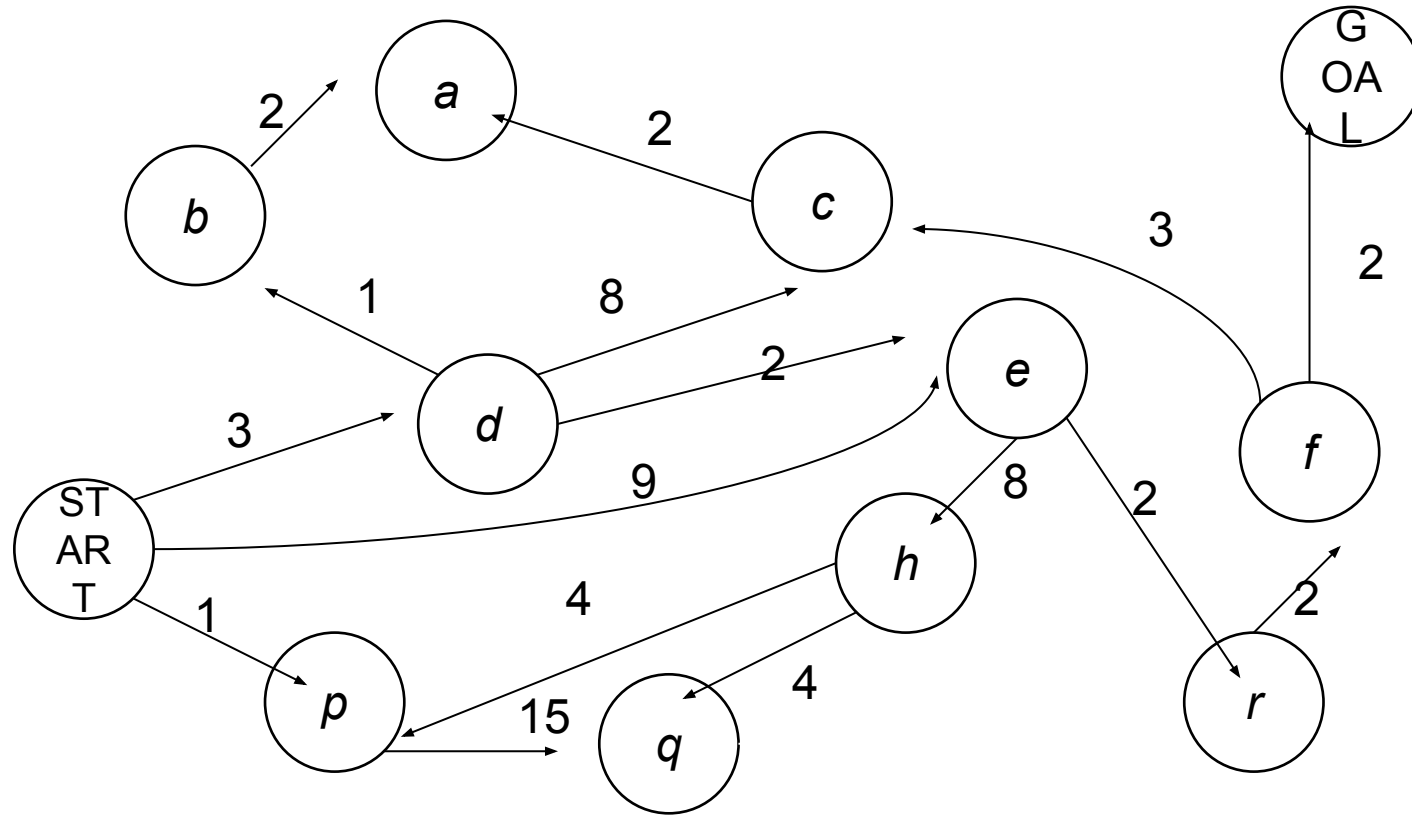


Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
 - Run a DFS with depth limit 1. If no solution...
 - Run a DFS with depth limit 2. If no solution...
 - Run a DFS with depth limit 3.
- Isn't that wastefully redundant?
 - Generally most work happens in the lowest level searched, so not so bad!

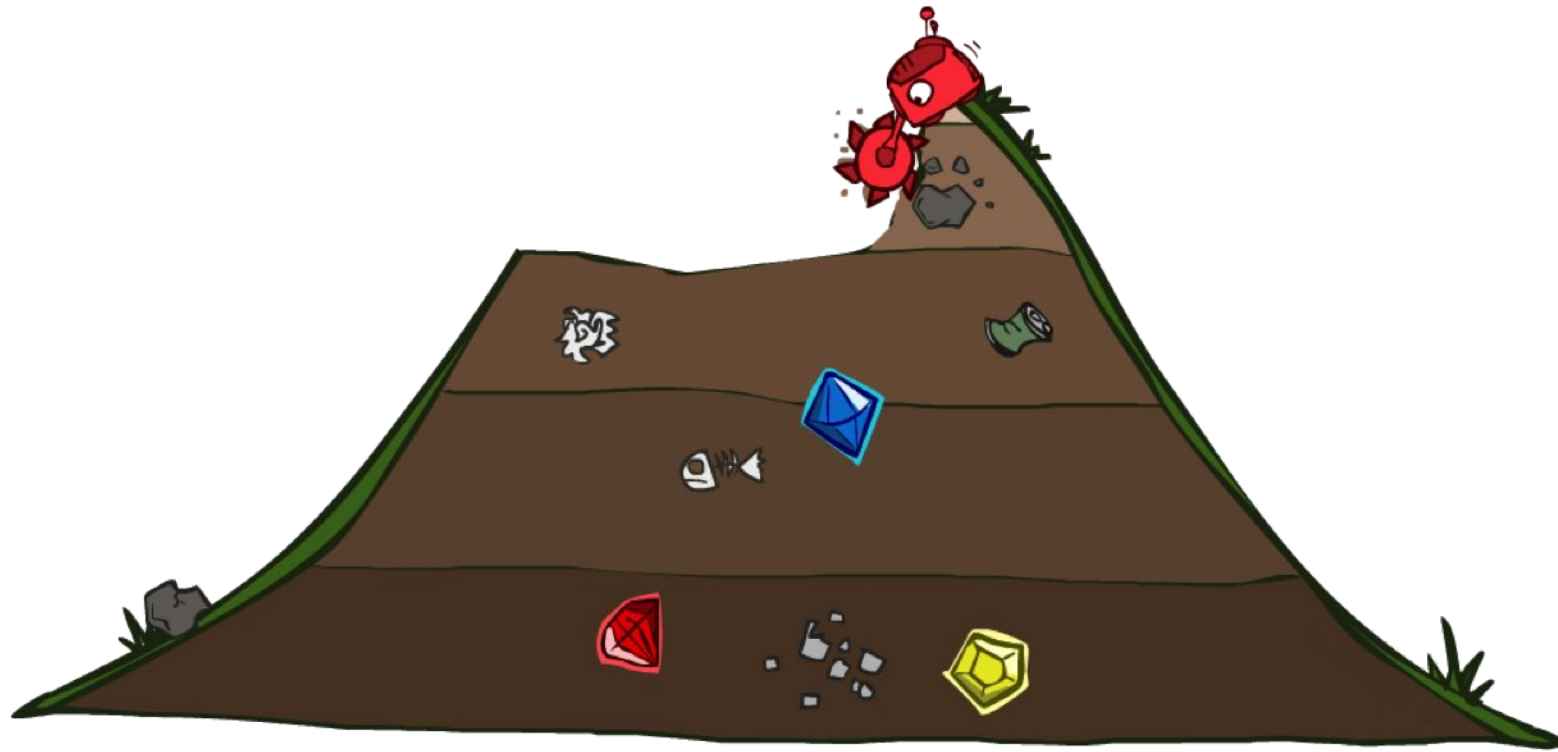


Cost-Sensitive Search



BFS always finds the path with the fewest steps, but not the one with the lowest total cost. Next, we will learn another search method that does find the path with the lowest cost, even if it has more steps.

Uniform Cost Search

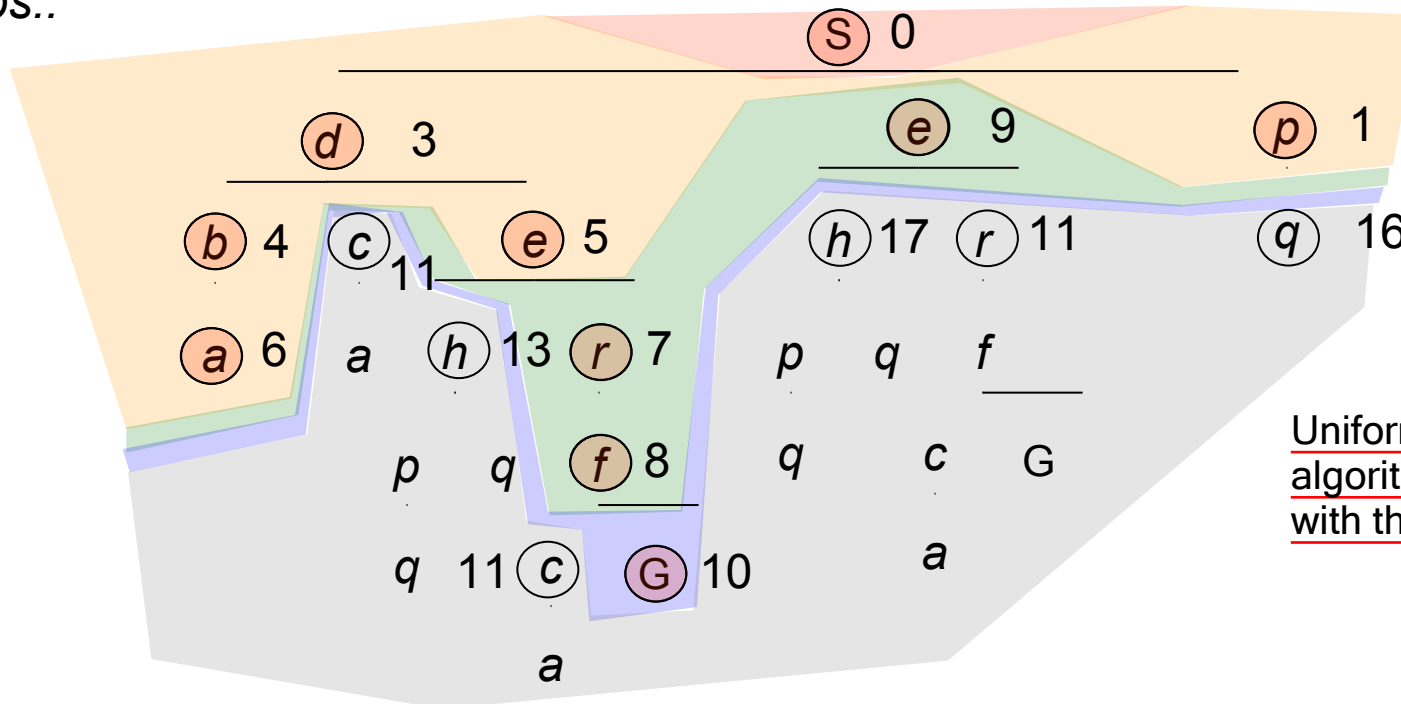
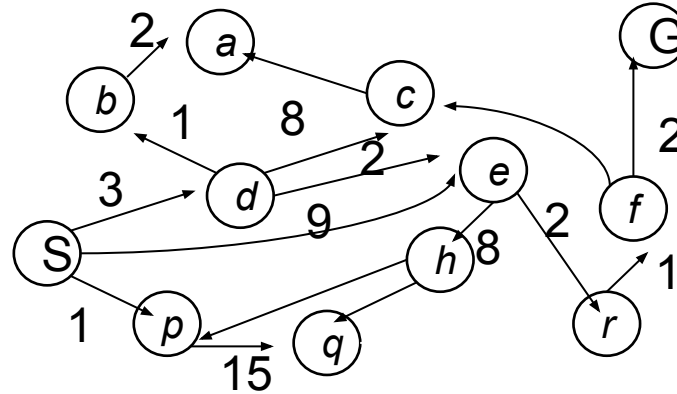


Uniform Cost Search

Strategy: expand a cheapest node first: This means UCS chooses the path that has cost the least so far, not the one that is shallowest or has the fewest steps..

Fringe is a priority queue (priority: cumulative cost)

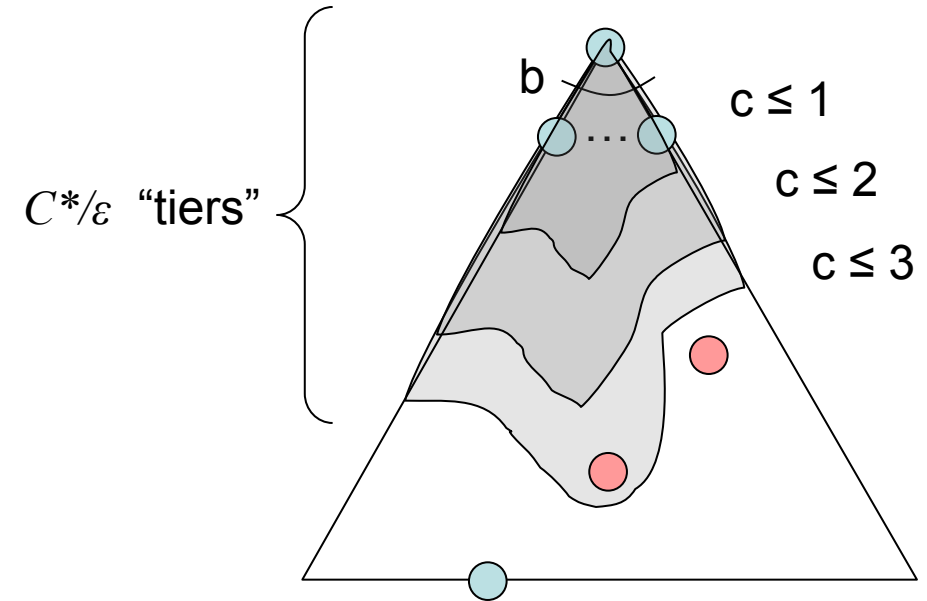
Cost contours



Uniform-Cost Search (UCS) is a search algorithm that always expands the node with the lowest total cost so far.

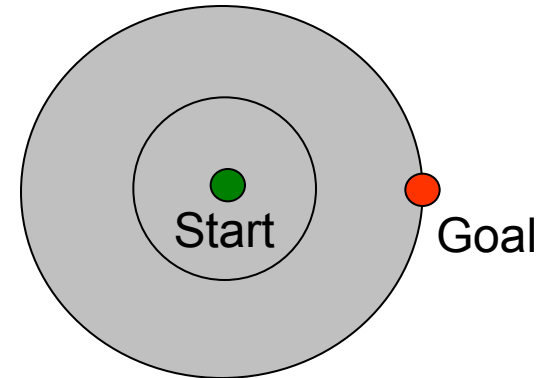
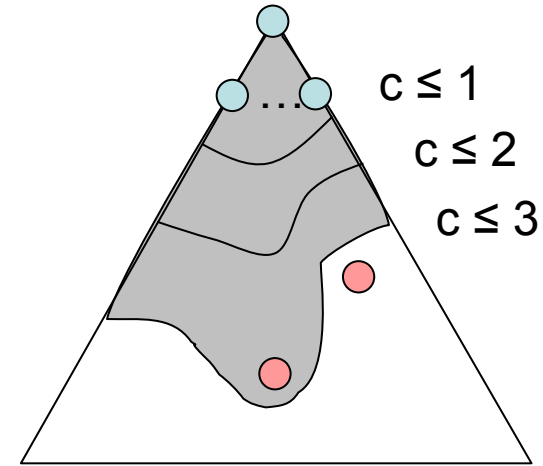
Uniform Cost Search (UCS) Properties

- What nodes does UCS expand?
 - Processes all nodes with cost less than cheapest solution!
 - If that solution costs C^* and arcs cost at least ε , then the “effective depth” is roughly C^*/ε
 - Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
 - Yes, as long as: the best solution has a finite cost, and every action has a positive cost (> 0). This ensures UCS doesn't loop forever over zero-cost cycles.
- Is it optimal?
 - Yes! (Proof next lecture via A^*)



Uniform Cost Issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
 - Explores options in every “direction”
 - No information about goal location
- We'll fix that soon!



[Demo: empty grid UCS (L2D5)]
[Demo: maze with deep/shallow
water DFS/BFS/UCS (L2D7)]

Video of Demo Empty UCS



Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 1)



Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 2)

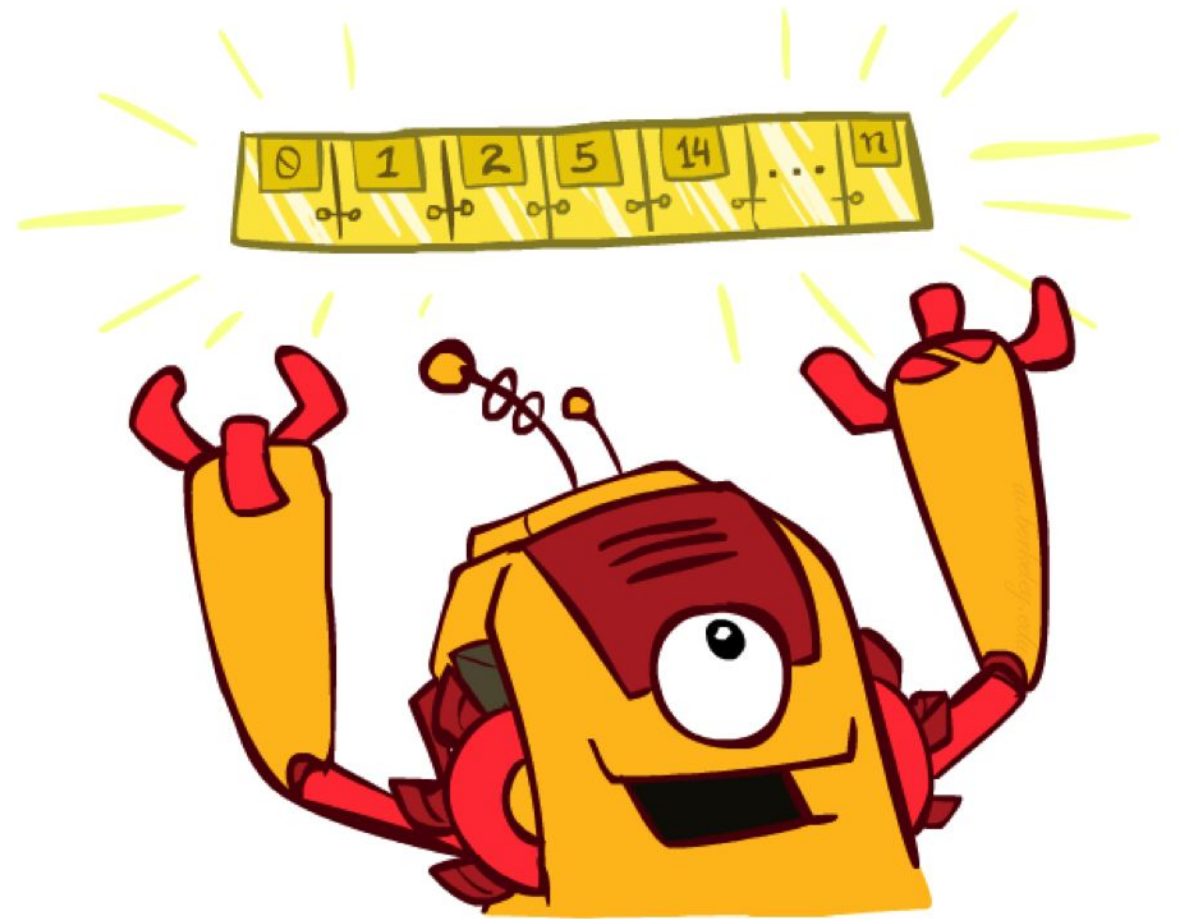


Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 3)



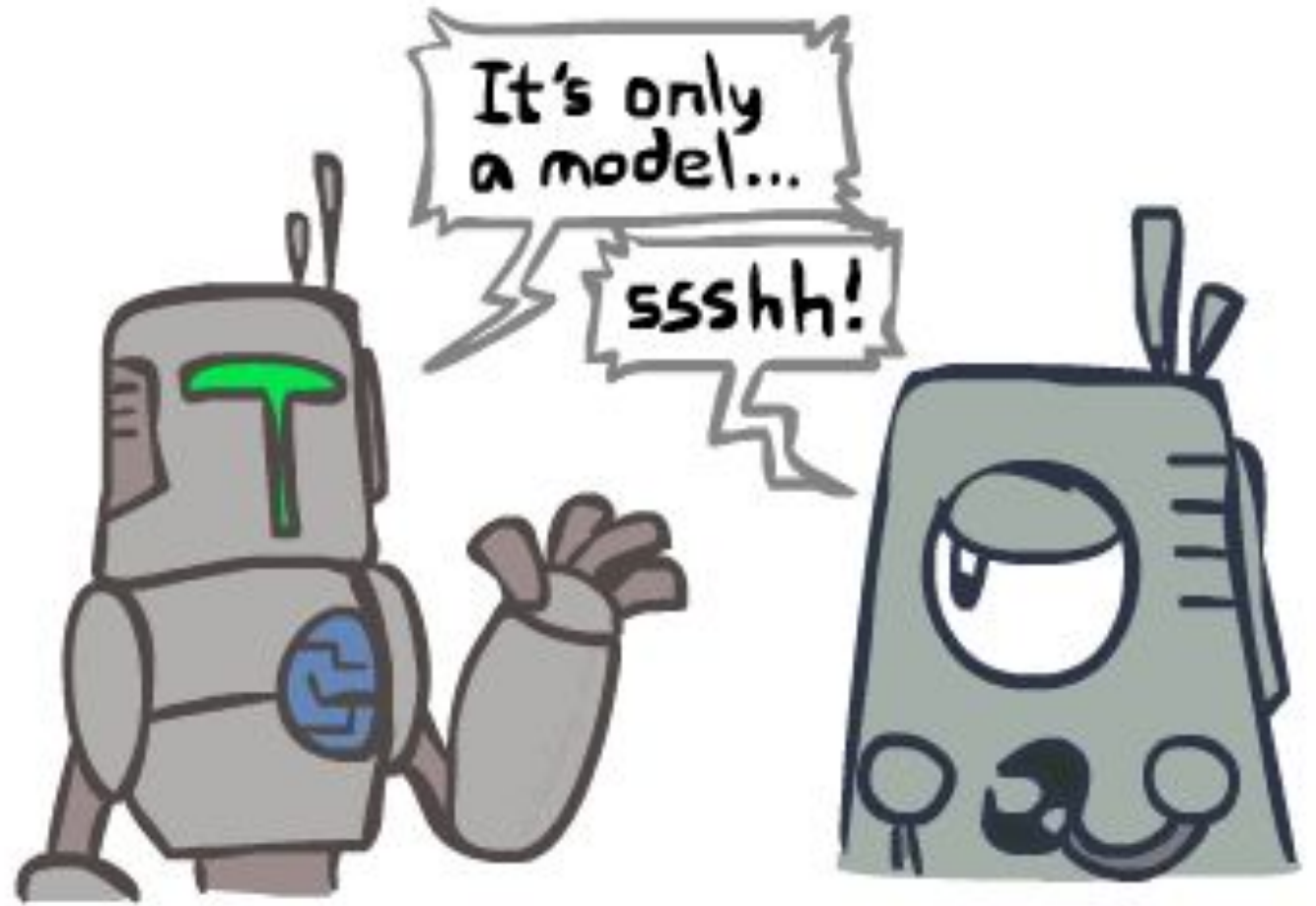
The One Queue

- All these search algorithms are the same except for fringe strategies
 - Conceptually, all fringes are priority queues (i.e. collections of nodes with attached priorities)
 - Practically, for DFS and BFS, you can avoid the $\log(n)$ overhead from an actual priority queue, by using stacks and queues
 - Can even code one implementation that takes a variable queuing object

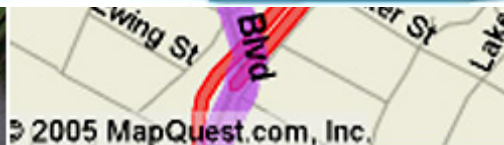
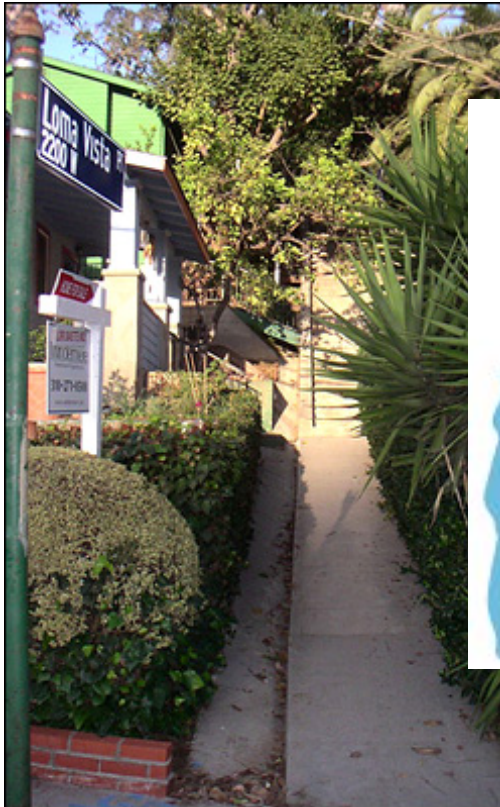


Search and Models

- Search operates over models of the world
 - The agent doesn't actually try all the plans out in the real world!
 - Planning is all “in simulation”
 - Your search is only as good as your models...



Search Gone Wrong?



Some Hints for P1

- Graph search is almost always better than tree search (when not?)

Ans: Graph search is usually better because it avoids revisiting states, saving time and preventing loops. However, it may be worse when memory is limited, because it must store all visited states. Tree search can be better in huge state spaces where storing the closed set is too expensive.

- Implement your closed list as a dict or set!
- Nodes are conceptually paths, but better to represent with a state, cost, last action, and reference to the parent node